

MUSE-Autoskill: Self-Evolving Agents via Skill Creation, Memory, Management, and Evaluation

Huawei Lin^{1,2,*}, Peng Li¹, Jie Song¹, Fuxin Jiang¹, Tieying Zhang^{1,†}

¹ByteDance Inc., ²Rochester Institute of Technology

*Work done during an internship at the ByteBrain team, †Corresponding author

Abstract

Large language model (LLM) agents rely on reusable skills to solve complex tasks. However, existing skill creation approaches treat skills as isolated and static artifacts, limiting their reusability, reliability, and long-term improvement. We propose **MUSE-Autoskill Agent (Memory-Utilizing Skill Evolution)**, a skill-centric agent framework that lets agents continuously improve their task-solving capability by creating, reusing, and refining skills under a unified lifecycle (creation, memory, management, evaluation, and refinement). Our framework enables agents to create skills on demand, store and reuse them across tasks, organize and select them efficiently, and evaluate them through unit tests and runtime feedback for continuous refinement. We further introduce skill-level memory that accumulates experience for each skill across tasks, enabling more effective reuse and adaptation over time. Experiments on SkillsBench provide initial evidence that lifecycle-managed skills can improve task success, efficiency, reuse, and cross-agent transfer, highlighting the importance of treating skills as long-lived, experience-aware, and testable assets.

Date: May 27, 2026

Correspondence: Tieying Zhang, tieying.zhang@bytedance.com

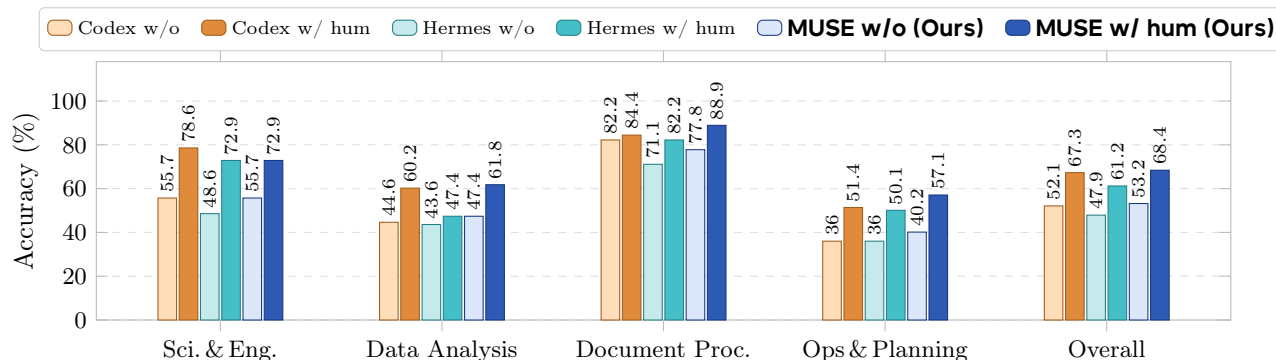


Figure 1 MUSE-Autoskill (ours) leads on SkillsBench across domains. Accuracy (%) of three GPT-5.5-backed agents on 51 SkillsBench tasks across four super-domains (Science & Engineering, Data Analysis, Document Processing, Ops & Planning) and the overall *Total*. Paired bars per agent: lighter = *without skills*, saturated = *with human skills*. MUSE-Autoskill achieves the highest with-skills score in 3 of 4 domains and on *Total* (68.4% vs. Codex 67.3% / Hermes 61.2%), a +15.2 pp lift consistent across agents. See Section 4 and Table 3.

1 Introduction

Skills for agents. Large language model (LLM) agents are increasingly tasked with solving complex, real-world problems that involve interacting with external tools, data, and code, often spanning many steps and disparate domains [8, 16, 35]. As task scope grows, raw model reasoning alone is insufficient: agents need access to reusable units of capability, namely skills, that encapsulate procedures, executable code, or domain-specific instructions and can be composed into solutions [2, 27]. Skills are emerging as the natural abstraction for scalable agent systems because they decouple capability from monolithic model weights, enabling modular execution and the accumulation of structured domain knowledge [2, 31]. The central open question is how to enable agents to continuously improve their capabilities through skills they can obtain, organize, and refine on their own, without relying on human authoring at every step.

Limits of AutoSkill. A growing line of work uses LLMs to synthesize skills automatically, starting from Voyager’s executable code library in Minecraft [27] and extending to general-purpose agents via AutoSkill [34], EvoSkill [1], and SkillGen [14]. More recent approaches use reinforcement learning to jointly optimize skill selection, use, and distillation (Skill1 [24]) or to train a dedicated skill curator (SkillOS [17]). On the production side, Anthropic’s Agent Skills [2] standardize skills as portable folders of instructions and scripts. While these methods successfully expand agent functionality, they typically cover only part of the skill lifecycle and leave four practical gaps: (i) a creation–usage mismatch, where skills are produced without access to the agent’s runtime context; (ii) no structured per-skill memory that accumulates free-form experience about individual skills across tasks; (iii) static, unvalidated skills without unit-test-driven evaluation or refinement; and (iv) poor context handling, where flat conversation histories truncate or overflow on long-horizon tasks.

Skill lifecycle. We argue that skills should not be one-off generation outputs but long-lived, evolving assets of an agent system. A useful skill is created on demand within the agent’s reasoning loop, stored with associated experience and metadata [18, 19, 26], retrieved when contextually relevant, validated through tests and runtime feedback, and continuously refined as new evidence accumulates [3, 14, 15]. We formalize this perspective as a unified skill lifecycle with five stages: **creation, memory, management, evaluation, and refinement**. This reframing turns skills from disposable artifacts into managed, testable, and transferable infrastructure: the foundation needed for agents to accumulate experience across tasks, sessions, and even across different agent systems.

MUSE-Autoskill framework. We instantiate this lifecycle in **MUSE-Autoskill Agent (Memory-Utilizing Skill Evolution; Figure 2)**. MUSE tightly couples skill creation with execution through a built-in `skill_create` tool invoked from within the runtime loop, eliminating the creation–usage mismatch. It introduces a multi-level memory comprising short-term, long-term, and (uniquely) skill-level memory, which accumulates per-skill experience across tasks and informs future invocations. An evaluation subsystem grounds reliability in unit tests and execution feedback, automatically triggering refinement when tests fail. A structured context manager with adaptive compression and cross-session state persistence supports long-horizon tasks without information loss or context-window blowup. Together, these components make skills externalized, testable, and transferable, rather than internal model behavior locked inside opaque weights.

Results. Figure 1 previews our headline results on **SkillsBench**, a benchmark of 51 real-world tasks graded by automated verifiers in standardized Docker environments. Among three GPT-5.5-backed agents, MUSE-Autoskill achieves the best *with-skills* accuracy in 3 of 4 super-domains and overall (**68.40%**, a +15.21 pp lift over its no-skills baseline). When MUSE-Autoskill creates skills from its own successful trajectories, accuracy on the 35 tasks where generation succeeds reaches **87.94%**, surpassing the human-skill ceiling. Generated skills also transfer cleanly: injected into a different agent (Hermes), they raise its accuracy by +10.51 pp, closing 79% of the gap to Hermes with human skills, evidence that MUSE produces externalized knowledge assets rather than agent-specific behavior tied to one runtime.

Contributions. This paper makes four contributions:

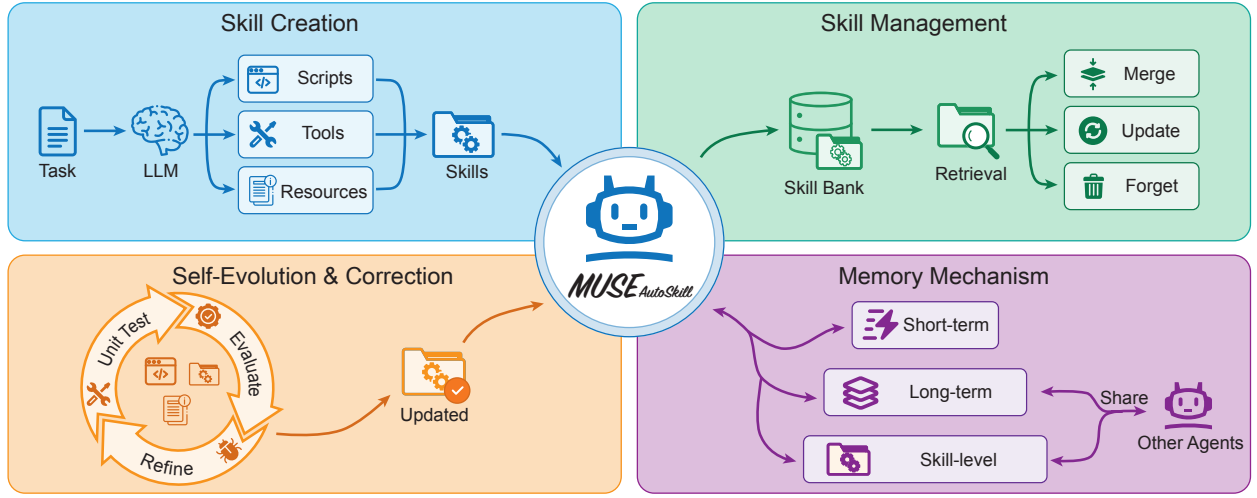


Figure 2 MUSE-Autoskill Agent architecture. MUSE organizes skills into a unified lifecycle of creation, memory, management, evaluation, and refinement, enabling agents to generate, refine, and reuse skills with accumulated experience over time.

- **Skill lifecycle.** We reframe skills from one-off generation outputs into long-lived, lifecycle-managed assets, identifying five stages (creation, memory, management, evaluation, refinement) that any practical skill-centric agent system must address.
- **MUSE-Autoskill.** A skill-centric agent that improves its task-solving capability over time by integrating skill creation with runtime execution, evaluating skills via unit tests and feedback, and automatically refining them when tests fail.
- **Infrastructure.** Multi-level memory with a novel skill-level memory that accumulates per-skill experience across tasks; adaptive context compression with cross-session state persistence; and cross-agent skill transfer that makes generated skills usable beyond their authoring agent.
- **Validation.** Best-in-class SkillsBench accuracy among three GPT-5.5-backed agents (68.40% with human skills, +15.21 pp lift); self-generated skills exceed the human-skill ceiling on 35 tasks (87.94%); generated skills transfer to a different agent with minimal loss.

2 Related Work

2.1 LLM Agents

LLM-based agents that interact with tools, environments, and data have advanced rapidly in recent years [5, 6, 22, 29]. Building on ReAct [35]’s interleaving of reasoning and action, follow-up systems extend the paradigm to broader workflows, including multimodal autonomous agents such as Agent-Omni [11] and OmniGAIA [10], and a wider body of work on self-improving agents [15, 26]. A parallel line of work focuses on equipping agents with tool-use capabilities, ranging from few-shot tool calling [21] to tool orchestration via model selection [23] and large-scale API retrieval [20]; for software engineering specifically, agents such as CodeAgent [36], SWE-Agent [32], and OpenHands [28] drive tool-integrated workflows over sandboxed shells and editors to resolve real-world repository tasks. The capabilities of such systems are now measured by general agent benchmarks including GAIA [16], SWE-bench [8], and AgentBench [13], which together cover web browsing, real-world software engineering, and multi-environment tool use. Despite this progress, most agent frameworks treat the set of available actions as either a fixed, hand-engineered tool registry or a flat conversational scratchpad. They do not natively support agents that can author, validate, and accumulate their own reusable capabilities over time, which is precisely the gap the skill-centric literature, and our framework, set out to close.

2.2 Automatic Skill Systems

We organize the growing literature on automatic skill systems along two axes: which stages of the skill lifecycle (creation, memory, management, evaluation, refinement) a method addresses, and whether it operates entirely at inference time or requires additional model training. Table 1 summarizes the resulting comparison along these two axes.

The first major direction builds skill systems on top of pretrained LLMs without any fine-tuning. Voyager [27] is the seminal example: in the Minecraft setting, it maintains an ever-growing library of executable-code skills, with self-verification and iterative prompting that lets the same LLM both author and refine skills in response to environment feedback. Follow-up work generalizes this paradigm to general-purpose agents: AutoSkill [34] derives, maintains, and reuses skills from dialogue and interaction traces as a model-agnostic plugin layer; EvoSkill [1] analyses execution failures and proposes new skills or edits, retaining only those that improve held-out validation under a Pareto-frontier selection; and SkillGen [14] iteratively refines skills via contrastive induction over successful and failed trajectories, modelling each skill as an intervention to empirically verify its net effect. The feedback-driven refinement underlying these methods is rooted in a broader self-improvement literature outside the skill setting: Reflexion [26] maintains reflective text in an episodic memory buffer across attempts, Self-Refine [15] iteratively rewrites outputs using self-generated critiques, Self-Debug [3] closes the loop on code generation with execution and unit-test traces, and ExpeL [37] extracts natural-language insights across training tasks for inference-time reuse. These methods all improve agent behavior through linguistic feedback but stop short of treating skills as first-class, externalized, testable artifacts that outlive a single task or agent. On the industrial side, Anthropic’s Agent Skills [2] standardize skills as portable folders of SKILL.md instructions and scripts loaded via progressive disclosure; this is the closest practical analogue of our externalized skill format, but the system leaves evaluation and refinement to human authoring. Collectively, these training-free methods are lightweight and naturally portable across LLM backbones, yet each covers only part of the lifecycle: none simultaneously supports structured per-skill memory, unit-test-driven evaluation, and automatic refinement triggered by test feedback.

A second, concurrent direction uses reinforcement learning to optimize skill behavior jointly with the policy. SkillMaster [33] learns a single policy that both acts and edits its skill bank, with edits credited by counterfactual downstream utility. Skill1 [24] frames skill evolution as a unified RL problem, co-optimizing skill selection, utilization, and distillation under a shared task-outcome reward. SkillOS [17] pairs a frozen executor with a trainable curator that updates an external skill repository from accumulated experience, and shows that the curator generalizes across executor backbones; this is a portability axis complementary to ours, where the skills themselves rather than the curator are the unit of transfer. Youtu-Agent [25] pursues a related direction via hybrid policy optimization of tools and agent configurations. These RL-based methods can attain strong optimality on the environments they are trained on, but they couple skill behavior to a trained policy or curator: migrating to a new backbone typically requires additional training, and skills produced by one trained policy are not directly usable by a different agent without re-training.

2.3 Benchmarks and Positioning

Several recent benchmarks complement the methods above by stressing different lifecycle stages. SkillsBench [9], which we adopt in our experiments, measures end-to-end task accuracy with and without skills across diverse Docker-evaluated real-world tasks. SkillRet [4] isolates the management stage by evaluating skill retrieval at scale from a library of nearly 18,000 community-contributed skills. SkillLearnBench [39] and LifelongAgentBench [38] focus on continual and lifelong skill acquisition over task streams, and notably report that strong individual methods do not consistently dominate, motivating system-level designs such as ours. A concurrent survey [31] catalogues skill-acquisition modalities and architectural choices for LLM agents, situating both training-free and training-based directions within a broader taxonomy.

Compared with the methods above, MUSE-Autoskill differs in that it brings all five lifecycle stages together within a single training-free framework, rather than addressing creation or refinement in isolation. In particular, it introduces skill-level memory that accumulates per-skill experience across tasks, uses unit-test-driven evaluation that automatically triggers refinement when tests fail, and is the only general-purpose method to empirically validate cross-agent skill transfer by injecting its generated skills into a different

Table 1 Related work on automatic skill systems by lifecycle stage. ✓ = covered; △ = partial; ✗ = not addressed. Memory = persistent per-skill experience across tasks. Cross-agent = skills from one agent are usable by another without modification; ✓ requires an explicit cross-agent transfer experiment, △ indicates portability only across LLM backbones or product variants of the same agent. Training-free = inference-time only, no fine-tuning or RL.

Method	Lifecycle stage					Cross-agent	Training-free
	Creation	Memory	Management	Evaluation	Refinement		
Voyager [27]	✓	✗	✓	△	✓	△	✓
AutoSkill [34]	✓	△	△	✗	✓	✗	✓
EvoSkill [1]	✓	✗	△	✓	✓	✗	✓
SkillGen [14]	✓	✗	✗	✓	✓	△	✓
Anthropic Skills [2]	✓	✗	✓	✗	✗	△	✓
SkillMaster [33]	✓	✗	△	△	✓	✗	✗
Youtu-Agent [25]	✓	✗	△	✗	✗	✗	✗
Skill1 [24]	✓	△	△	△	△	✗	✗
SkillOS [17]	✓	✓	✓	△	✓	✗	✗
MUSE-Autoskill (Ours)	✓	✓	✓	✓	✓	✓	✓

agent without modification (Section 4); other portability claims in the literature are limited to swapping the underlying LLM backbone or sharing skills across product variants of the same agent family, without an explicit cross-agent experiment. The combination of full lifecycle coverage and a training-free design also makes the system portable across LLMs and agent architectures, as summarized in the bottom row of Table 1.

3 MUSE-Autoskill Agent

In this section, we present **MUSE-Autoskill Agent**, a skill-centric agent framework that solves complex tasks by dynamically creating, reusing, and refining skills. MUSE integrates skill creation, execution, memory, management, and evaluation within a unified agent loop. Figure 2 illustrates the overall architecture and the five lifecycle stages described below.

3.1 Agent Framework

The agent operates in an iterative decision-making loop consisting of three core stages: *Planning*, *Action*, and *Observation* [35]. Given an input query, the agent continuously cycles through these stages to progressively solve the task. This design enables dynamic reasoning, skill invocation, and adaptive refinement based on intermediate feedback.

Planning In the planning stage, the agent interprets the input query and determines the next step toward achieving the task objective. This involves decomposing the problem, selecting appropriate strategies, and deciding whether to invoke external skills. The agent may also leverage past observations and memory to refine its plan, enabling more informed and context-aware decisions.

Action In the action stage, the agent executes the planned step by invoking skills. These may include retrieving existing skills from the skill bank or utilizing built-in functions such as skill creation and web search. The selected skill is invoked within the agent’s ReAct loop using its built-in tools, producing intermediate or final outputs for the task. The detailed execution mechanism of skills will be introduced in Section 3.2.

Observation In the observation stage, the agent collects and analyzes the results returned from execution. These observations are used to evaluate progress toward the goal and to inform subsequent planning decisions. Through this feedback loop, the agent can iteratively refine its behavior, handle errors, and adapt to complex, multi-step tasks.

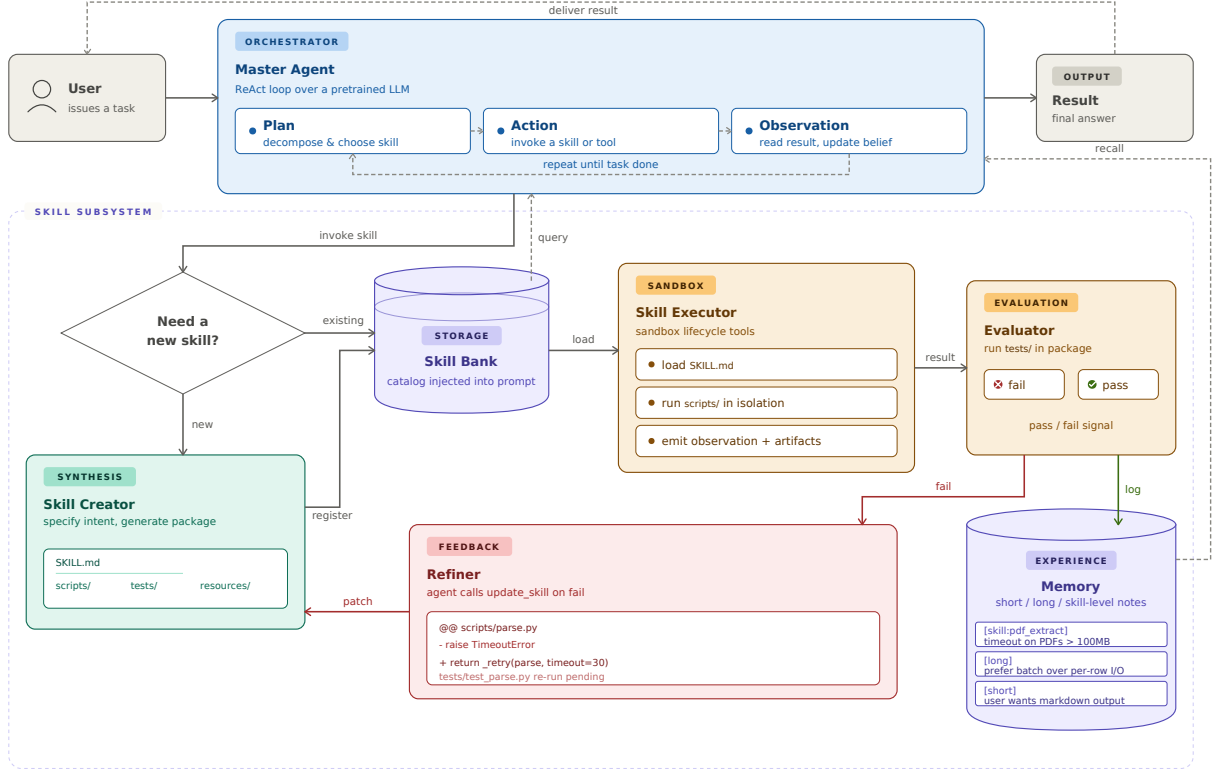


Figure 3 End-to-end flow of MUSE-Autoskill. The Master Agent runs a ReAct loop; when a skill is needed it either retrieves one from the Skill Bank or dispatches the Skill Creator to synthesize a new package (SKILL.md plus optional scripts/ and tests/). The Evaluator runs the bundled tests; on **pass**, observations are appended to Memory and surfaced on later steps; on **fail**, the Refiner patches the package and re-enters the loop.

3.2 Skill Lifecycle

As illustrated in Figure 3, the agent organizes skills into a unified lifecycle of five stages: *creation*, *memory*, *management*, *evaluation*, and *refinement*. To bootstrap this process, the agent is equipped with a small set of built-in skills, including `skill_create` and `web_search`. All other skills are not predefined but must be created through this mechanism, ensuring that the agent’s capabilities are dynamically constructed and continuously evolving.

Skill As illustrated in Figure 2, a *skill* is the basic unit of execution in our system. Each skill is packaged as a structured directory with standard components, following Anthropic’s Agent Skills format [2]. It includes a `SKILL.md` file that defines its interface, such as name, description, inputs, and outputs, and may also include subdirectories like `scripts/` for executable code, `resources/` for auxiliary data, and `tests/` for validation.

Skills are executed through a unified interface. At runtime, the agent reads `SKILL.md` to understand how to use the skill, and decides whether to read resources, run scripts, or both. If scripts are required, the execution engine runs the corresponding code with the given inputs and returns the outputs.

Using skills improves efficiency. Instead of generating detailed reasoning steps every time, the agent can call a skill with a short interface, which reduces token usage. Skills can also be reused across tasks, allowing the agent to avoid repeating work and making the system more scalable over time.

Skill Creation As illustrated in Figure 2, new skills are generated through the built-in `skill_create` skill. When existing skills are not sufficient, the agent provides a high-level specification of the desired functionality, including its purpose, inputs, and expected outputs.

Based on this specification, the system follows a structured pipeline to construct the skill. It first generates

the `SKILL.md` file to define the interface, then plans the internal structure such as `scripts/`, `resources/`, and `tests/`, and finally generates the corresponding files. The result is a complete and executable skill package.

After creation, each skill is gated by an evaluation step: the system runs the unit tests in the newly written `tests/` directory inside the sandbox, and only registers the skill into the Skill Bank if all tests pass. If tests fail, the agent inspects the error trace and invokes `update_skill` to patch the package before re-running tests. This create → evaluate → register loop ensures only reliable skills enter the bank and are reusable in future tasks. This design also keeps all non-built-in functionality consistently created as skills, making them easy to reuse, validate, and improve over time.

Skill Evaluation As illustrated in Figure 2, skills are evaluated to ensure their correctness and reliability before being reused. This evaluation is primarily performed through unit tests defined in the `tests/` directory of each skill. After a skill is created, the system executes these tests with predefined inputs and verifies whether the outputs match expected results.

This process filters out incorrect or unstable skills and provides signals for further refinement. As part of the self-evolution loop shown in Figure 2, failed tests can trigger updates or regeneration of the skill. By enforcing systematic evaluation, the agent maintains a high-quality skill set and ensures robust performance during execution.

Skill Execution As illustrated in Figure 2, skill execution is carried out within the agent’s ReAct loop using its built-in tools. Given a task, the agent reads the available skill catalog and selects an appropriate skill. It then reads the `SKILL.md` file to understand the skill interface, standard operating procedure, and required components.

Following the procedure defined in `SKILL.md`, the agent decides whether to read from `resources/`, execute code in `scripts/` via sandbox tools, or combine both. Code execution is mediated by a small set of sandbox lifecycle tools (`create_sandbox`, `sandbox_run`, `sandbox_upload/sandbox_download`, and `close_sandbox`) that the agent invokes from inside its ReAct loop. Each sandbox is an isolated process / container with its own filesystem, so failures, side effects, and resource usage are contained per skill invocation. Rather than introducing a separate execution engine, skill execution reuses the same general-purpose tools the agent already uses (file reading, terminal commands, sandbox calls), which avoids redundant infrastructure and lets execution benefit from the agent’s full reasoning capability.

The execution process is iterative: intermediate results are fed back into the agent’s reasoning loop, enabling progressive refinement and error handling. This unified approach ensures consistent execution across all skills while preserving flexibility for both simple and complex tasks.

Skill Memory As illustrated in Figure 2, the agent maintains memory at multiple levels to support skill reuse and accumulation over time. In particular, skill-level memory stores the skills themselves along with their metadata, such as descriptions, inputs, and usage history. This allows the agent to efficiently retrieve relevant skills for new tasks.

In addition, the agent appends notes and observations to short-term and long-term memory, providing context for future decisions. This memory helps the agent avoid redundant skill creation, reuse effective solutions, and improve performance over time. By maintaining structured memory around skills, the system enables continuous learning and more efficient task execution.

Skill Management As illustrated in Figure 2, skill management maintains the quality and usability of the skill bank. Each skill is indexed using metadata from `SKILL.md`, including its name, description, inputs, and outputs. At the start of each task, the agent is provided with a catalog of available skills injected into the system prompt, following the progressive-disclosure pattern of Anthropic’s Agent Skills [2]. The agent then selects the most relevant skill during planning based on this catalog, enabling efficient reuse and reducing unnecessary skill creation.

In addition to retrieval, the system supports continuous maintenance of the skill bank through three mechanisms: *refinement*, *merging*, and *pruning*. When a skill fails unit tests or produces incorrect outputs during execution, the agent revises or regenerates it based on the error feedback. When newly created skills overlap significantly with existing ones, the agent merges them into a single, more general skill to avoid redundancy. Skills that consistently fail or remain unused over time are pruned from the skill bank. Together, these mechanisms keep the skill bank compact, reliable, and scalable as the agent accumulates more skills over time.

3.3 Memory

Memory plays a central role in enabling MUSE to accumulate knowledge and reuse previously acquired capabilities. Our design builds on prior hierarchical memory architectures for LLM agents: MemGPT [18] pages between in-context and external memory in an OS-style hierarchy, Generative Agents [19] maintain a memory stream with periodic synthesis into higher-level reflections, and Reflexion [26] and ExpeL [37] accumulate natural-language reflections and insights across episodes. MUSE extends these by adding a per-skill memory scope tied to each `SKILL.md` file, complementing short- and long-term layers shared with prior work.

Skill-level Memory Each skill in the bank carries its own `.memory.md` file, into which the agent appends notes, lessons, and usage observations accumulated across tasks (e.g., known failure modes, input format quirks, performance caveats). When the same skill is loaded later, this per-skill memory is surfaced alongside its `SKILL.md` interface, letting the agent benefit from previously learned experience without re-deriving it.

Short-term Memory Short-term memory maintains the current task context, including intermediate reasoning steps, observations, and temporary execution results. As the context grows, it is adaptively compressed by summarizing intermediate steps, allowing the agent to handle long-horizon tasks without exceeding the model’s token budget.

Long-term Memory Long-term memory stores persistent notes the agent appends across sessions, including reusable conclusions, environment quirks, and general lessons learned outside any single skill (e.g., “prefer batched I/O,” “the project uses pinned package versions”). Unlike short-term memory, long-term memory is not subject to compression and serves as a growing repository of accumulated experience, enabling the agent to improve decision-making over time by drawing on lessons learned in prior runs.

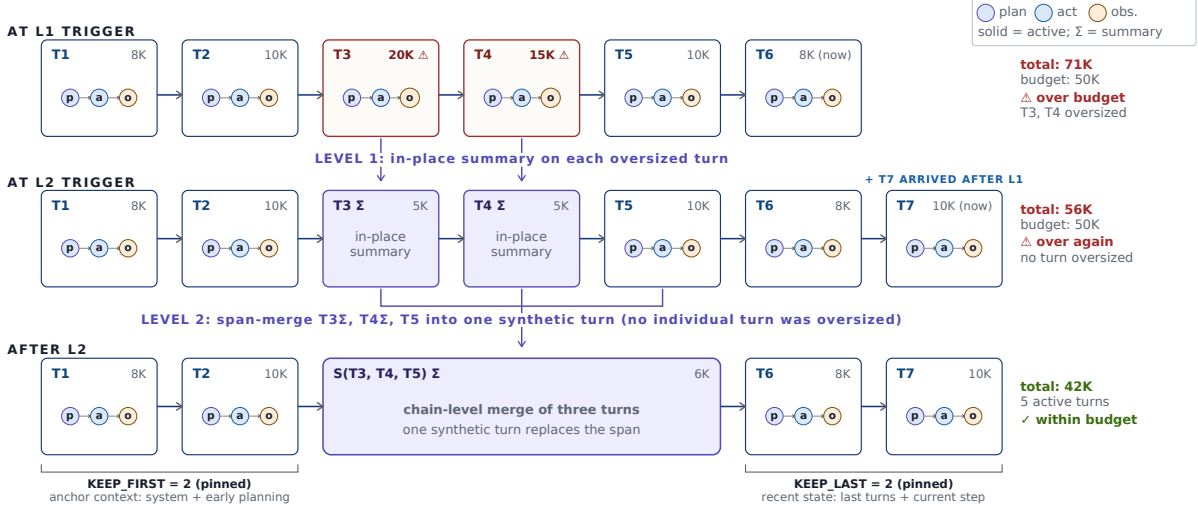
3.4 Context Management

The agent maintains context as a DAG of conversation nodes, one per turn (Figure 4). Each node records the model response, tool calls, observations, and per-call token usage from one step. Every node carries two sets of pointers: a mutable `parent_id` that defines the current active chain sent to the LLM, and an immutable `history_prev/history_next` pair that defines the full history of original turns. The active chain is always a sub-graph of the full history.

As tasks grow longer, the accumulated short-term context can exceed the model’s token budget. Existing remedies span token-level prompt compression [7], attention-sink-based KV retention for streaming inference [30], and OS-style virtual context management for general LLM agents [18]; positional studies further document significant degradation when relevant content is buried in the middle of a long context [12], which motivates the explicit first/last pinning we adopt below. To handle this at the agent level, MUSE applies adaptive context compression with two levels. **Level-1** (single-node compression) scans the active chain for individual nodes whose token footprint exceeds a per-node threshold (typically a large tool output or a verbose observation) and replaces that node’s content with a compact summary while keeping it in the chain. If the total context is still over budget after Level-1, **Level-2** (chain compression) merges a contiguous range of intermediate nodes into a single synthetic summary node, which then takes the place of those nodes in the active chain. We always try Level-1 first because it is the strictly less destructive operation: only the offending node’s payload is rewritten, while the per-turn boundaries and the full plan/action/observation structure of the chain are preserved, so downstream turns can still reference earlier turns by their original positions. Level-2 collapses several turns into one synthetic node and loses that per-turn structure, so we

Context management: DAG of ReAct turns with two-level adaptive compression

each turn is a (plan, action, observation) triple chained left-to-right; L1 rewrites individually oversized turns; L2 merges the compressible span when L1 is not enough



All compression operates on the active chain only; the original 7 turns remain in the full history (linked by immutable `history_prev` / `history_next` pointers), so any prior state can be replayed or resumed across sessions.

Figure 4 Adaptive context compression over a DAG of ReAct turns. Each turn is a (plan, action, observation) triple; the first `KEEP_FIRST` and last `KEEP_LAST` turns are always pinned and only the middle is eligible for compression. **Top→Middle:** Level-1 rewrites individually oversized turns in place. **Middle→Bottom:** when no single turn is oversized but the chain is still over budget, Level-2 merges the compressible span into one synthetic node. Original turns remain in the full history (linked by immutable `history_prev`/`history_next` pointers), so the trajectory is fully replayable.

invoke it only when single-node summaries alone cannot bring the chain under budget. In both levels the original nodes remain in the full history, so the active chain is always recoverable. Long-term memory and the skill bank, by contrast, are stored separately and are not subject to compression, allowing the agent to accumulate experience across sessions without loss.

In addition, the agent’s full state, including conversation history, skill usage records, and execution metadata, is persisted as a snapshot after each session. This allows tasks to be resumed from an intermediate state without restarting from scratch, which is essential for complex, long-horizon workflows that may span multiple sessions.

4 Experiments

We conduct experiments on SkillsBench to evaluate three aspects of our framework: whether skill usage improves agent performance, whether MUSE-Autoskill can automatically generate effective skills from its own experience, and whether generated skills can transfer across agents.

4.1 Experimental Setup

Benchmark We evaluate on **SkillsBench** [9], a benchmark designed to assess AI agents on real-world tasks that require domain-specific knowledge and tool use. Each task runs in an isolated Docker container and is graded by an automated verifier that checks only the final output files, assigning a reward in $[0, 1]$. We evaluate on 51 selected tasks spanning four super-domains: science & engineering (scientific computing and simulation), data analysis, document processing, and ops & planning (system operations and planning/optimization). Tasks are selected such that all participating agents complete them without Docker environment failures, enabling direct cross-agent comparison. The full task list is provided in Appendix A.

Table 2 Accuracy (%) on 51 SkillsBench tasks (macro-average over 5 runs per task). Human skills consistently improve all agents. **Bold** = best in column; **blue row** = MUSE-Autoskill (ours).

Agent	Without Skills	With Human Skills	Lift
Codex	52.11%	67.28%	+15.17%
Hermes	47.89%	61.21%	+13.33%
MUSE-Autoskill (Ours)	53.19%	68.40%	+15.21%

Agents and Models We evaluate three agents, all using GPT-5.5 as the backbone model: **MUSE-Autoskill** (our method), **Codex**, and **Hermes**. Since all agents share the same underlying model, performance differences reflect agent system design (including tool strategies, context management, planning, and skill usage) rather than model capacity.

Evaluation Protocol Each agent–task–configuration combination is run 5 times independently in separate Docker containers. We compute a task-level score by averaging the 5 rewards, then report the macro-average over all 51 tasks (each task weighted equally). Runs that fail due to environment errors are excluded; runs where the agent produces no output are counted as reward 0. For the skill generation experiment, tasks where no skill is generated are counted as 0 in the denominator rather than being excluded from the macro-average.

4.2 Effect of Skill Usage

Setup We compare each agent under two conditions: *without skills* (the agent relies solely on its own knowledge) and *with human skills* (domain-specific skills authored by SkillsBench task designers are injected into the agent’s workspace at task start). All other settings are identical.

Results Table 2 shows the results across all three agents. All agents benefit substantially from human skills, with accuracy gains of 13–15 percentage points. MUSE-Autoskill achieves the highest accuracy both with and without skills, reaching **68.40%** with human skills.

Discussion The consistent improvement across all agents suggests that the skill mechanism itself is effective. The performance gap between MUSE-Autoskill and the other agents with human skills suggests that MUSE-Autoskill is better at reading, interpreting, and applying skill content within its reasoning loop than the other two agents we tested.

Per-Domain Breakdown SkillsBench’s per-task **category** metadata is free-text (41 distinct labels for 51 tasks), so we group tasks into four balanced super-domains: **Science & Engineering** (S&E, 14 tasks; scientific computing + engineering simulation), **Data Analysis** (DA, 15), **Document Processing** (DP, 9), and **Ops & Planning** (O&P, 13; system operations + planning/optimization). Figure 1 and Table 3 report macro-average accuracy in each domain under both conditions. MUSE-Autoskill achieves the highest *with-skills* score in 3 of 4 domains; in S&E it trails Codex by 5.7% due to three boundary failures (*lake-warming-attribution*, *flood-risk-analysis*, *radar-vital-signs*) where the verifier penalizes methodology choices that are not pinned down by the task spec.

4.3 Automatic Skill Generation

Setup We evaluate MUSE-Autoskill’s ability to generate reusable skills from its own successful trajectories. The process follows a two-phase protocol. In **Phase 1**, MUSE-Autoskill solves each task without any skills (5 runs), using the same runs as the without-skills condition above. For tasks where at least one run succeeds, we select the best trajectory and invoke the *skill_create* tool to distill it into a **SKILL.md** and optional helper scripts. In **Phase 2**, the generated skill is injected back and the agent is re-evaluated (5 runs). Tasks where Phase 1 produces no successful run cannot generate a skill and are counted as 0 in the overall average.

Table 3 Per-domain accuracy (%) under *without skills* (w/o) and *with human skills* (w/ hum) conditions. **Bold** = best in each row’s *with-skills* columns; **blue columns** = MUSE-Autoskill (ours). MUSE-Autoskill achieves the best with-skills score in 3 of 4 domains.

Domain	#	Codex		Hermes		MUSE-Autoskill (Ours)		Best
		w/o	w/ hum	w/o	w/ hum	w/o	w/ hum	
Science & Engineering	14	55.71	78.57	48.57	72.86	55.71	72.86	Codex
Data Analysis	15	44.64	60.22	43.61	47.39	47.39	61.78	Ours
Document Processing	9	82.22	84.44	71.11	82.22	77.78	88.89	Ours
Ops & Planning	13	36.00	51.38	36.00	50.08	40.15	57.08	Ours
Macro-average	51	52.11	67.28	47.89	61.21	53.19	68.40	Ours

Table 4 Accuracy (%) of MUSE-Autoskill under different skill conditions. Self-created skills yield a +7.16% gain over the no-skill baseline. On the 35 tasks where a skill was successfully generated, Phase 2 accuracy reaches 87.94%, surpassing the human-skill reference. **Bold** = best; **blue row** = our self-created skills (the row we report as the main result).

Configuration	Accuracy (51 tasks)
MUSE-Autoskill without skills (baseline)	53.19%
MUSE-Autoskill with human skills (reference)	68.40%
MUSE-Autoskill self-created skills (Ours)	60.35%

Results Of the 51 tasks, MUSE-Autoskill successfully generates skills for **35** (68.6%). Table 4 compares overall accuracy across configurations. Per-task Phase 2 accuracy for all 35 tasks is provided in Appendix J.

Discussion On the 35 tasks with generated skills, Phase 2 accuracy reaches **87.94%**, which exceeds the human-skill ceiling of 68.40%. This indicates that skills distilled from real successful trajectories can encode highly task-relevant knowledge. The overall 51-task score of 60.35% is lower because the 16 tasks where Phase 1 entirely fails contribute 0% and are included in the denominator. The primary bottleneck is therefore coverage (the agent’s ability to solve tasks without skills) rather than the quality of the skills it generates.

4.4 Cross-Agent Skill Transfer

Setup We test whether MUSE-Autoskill’s generated skills can benefit a different agent. We inject the same generated skill files from the previous experiment into Hermes without any modification, and evaluate Hermes on the 51 tasks (5 runs each). For the 16 tasks without a generated skill, we reuse Hermes’s existing without-skills run results (all 0) rather than running new trials.

Results Table 5 summarizes the results. Hermes improves from 47.89% to **58.40%** (+10.51%) when using MUSE-Autoskill’s generated skills, closing 79% of the gap to Hermes with human skills. Notably, Hermes (58.40%) approaches MUSE-Autoskill (60.35%) when both use the same generated skills, with only a 1.95-point residual. Task-level comparisons between generated and human skills are provided in Appendix J.

Discussion Hermes and MUSE-Autoskill end within ~2 pp of each other when using the same generated skills (58.40% vs. 60.35%), showing that the skill content is not tailored to MUSE-Autoskill’s internal behavior. Generated skills are externalized as readable documents and scripts, making them genuinely transferable knowledge assets across agents and architectures.

Skill Generation and Usage Cost Table 6 reports the one-time cost of generating a skill (Phase 2) and the per-task cost of using a skill, on the 35 tasks where MUSE-Autoskill produced a skill. Generating a skill costs ~383K tokens and ~164s of agent time per task — roughly $\frac{2}{3}$ of one no-skill run, paid once. Using the generated skill is then cheaper, not more expensive, than using a human skill: MUSE-Autoskill drops from

Table 5 Cross-agent transfer results on 51 tasks. Hermes using MUSE-Autoskill generated skills closes 79% of the gap to human skills, and recovers most of MUSE-Autoskill’s accuracy on the same skills (within ~ 2 pp). **Bold** = best in row; **blue column** = MUSE-Autoskill (ours).

Configuration	Hermes	MUSE-Autoskill (Ours)
Without skills	47.89%	53.19%
With MUSE-Autoskill generated skills	58.40%	60.35%
With human skills (reference)	61.21%	68.40%

Table 6 Skill generation and usage cost (median per task; 35 tasks where MUSE generated a skill). Tokens = input plus output, summed across all API calls per run, then median across runs and tasks; latency excludes verifier and Docker setup. **Blue rows** = MUSE-Autoskill generated skills (ours).

Agent	Configuration	Tokens	Latency (s)	Turns
One-time skill generation cost				
MUSE-Autoskill	Phase 2 (skill creation)	383K	164	7
Per-task usage cost (35 tasks)				
MUSE-Autoskill	Without skills	578K	684	20
MUSE-Autoskill	With human skills	615K	656	19
MUSE-Autoskill	With generated skill (ours)	493K	411	15
Hermes	Without skills	181K	370	14
Hermes	With human skills	186K	369	14
Hermes	With generated skill (ours)	97K	257	13

615K tokens / 656s with human skills to 493K tokens / 411s with its generated skill (-20% tokens, -37% latency, $19 \rightarrow 15$ turns), and Hermes drops from 186K tokens / 369s to 97K tokens / 257s (-48% tokens, -30% latency). This is the opposite of what `SKILL.md` length alone would predict (Figure 6: MUSE skills are $2.2\times$ longer than human skills): the extra procedural detail replaces a longer noisy reasoning trajectory with a tighter procedure, so the agent finishes in fewer turns overall. Amortized against the 122K-token saving per use, the 383K generation cost breaks even after ≈ 3 reuses for MUSE-Autoskill, and the latency saving (~ 245 s/use vs. human, ~ 273 s/use vs. no-skill) already exceeds the 164 s creation cost on the very first reuse.

4.5 Generated Skill Capabilities

Aggregate Performance Figure 5 plots mean reward against two cost axes on the 35 tasks where MUSE-Autoskill produces a skill: median per-task latency (panel a) and median per-task tokens (panel b). Both panels overlay MUSE-Autoskill (blue) using its own generated skills and Hermes (teal) using those same MUSE-generated skills, under three conditions each (without skills, with human skills, with MUSE-generated skills). For both agents and on both axes, the with-generated-skill point is the unique Pareto-optimal configuration: higher reward AND lower latency AND fewer tokens than either without skills or with human skills. For MUSE-Autoskill, generated skills lift mean reward by $+11.0$ pp ($76.9\% \rightarrow 87.9\%$, vs. only $+7.9$ pp with human skills), while cutting median latency by 273 s ($684 \rightarrow 411$ s) and median tokens by 85K ($578 \rightarrow 493$ K). For Hermes, the lift from MUSE-generated skills is $+15.3$ pp ($69.8\% \rightarrow 85.1\%$, closing 84% of the gap to MUSE-Autoskill on these 35 tasks), at 113 s lower latency ($370 \rightarrow 257$ s) and 84K fewer tokens ($181\text{K} \rightarrow 97\text{K}$) — in fact Hermes with MUSE-generated skills outperforms Hermes with human skills on all three axes ($85.1\% / 257\text{ s} / 97\text{K tokens}$ vs. $77.8\% / 369\text{ s} / 186\text{K tokens}$). On the 35 tasks the lift is driven by a handful of cases where the baseline was stuck below 60% (e.g., `flink-query`, `protein-expression-analysis`, and `weighted-gdp-calc` jump $20\% \rightarrow 100\%$; `adaptive-cruise-control` jumps $40\% \rightarrow 100\%$); on the remaining tasks the skill does not introduce errors. Three tasks regress (e.g., `hvac-control`, $80\% \rightarrow 20\%$); we discuss the cause as case study (iv) below.

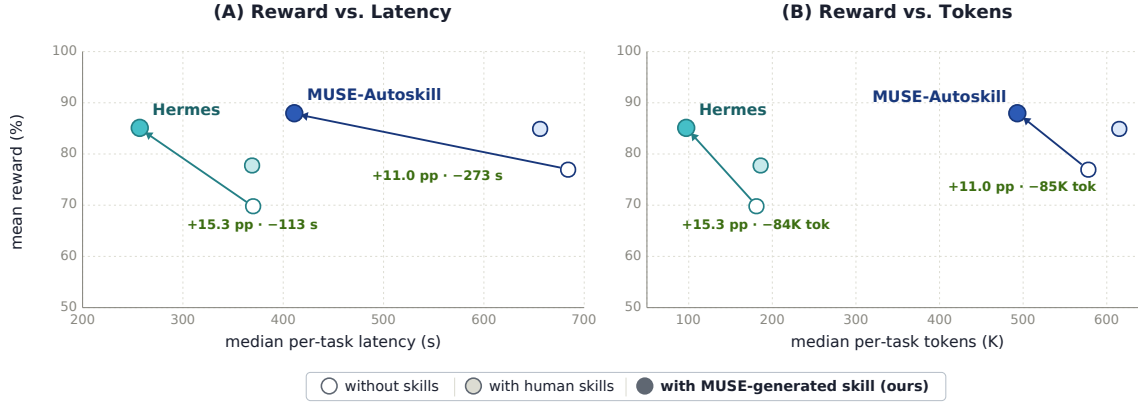


Figure 5 Generated skills are Pareto-optimal: higher reward, lower latency, and fewer tokens than human skills (mean over 35 tasks where MUSE-Autoskill generated a skill, 5 runs per task). **(A)** Mean reward vs. median per-task latency. **(B)** Mean reward vs. median per-task tokens. Both panels show MUSE-Autoskill (blue) using its own generated skills and Hermes (teal) using those same MUSE-generated skills with no modification. Open circle = without skills; light fill = with human skills (reference); solid fill = with MUSE-generated skills (ours). Arrows show the shift from without skills to with MUSE-generated skill for each agent; the inline label gives $(\Delta \text{reward} \cdot \Delta \text{cost})$. Full per-task numbers are in Appendix J.

Case Studies We highlight three skills where the generated artifact carries non-trivial domain knowledge, plus one regression.

(i) *adaptive-cruise-control* requires a discrete PID controller satisfying verifier constraints on overshoot, steady-state error, and rise time. MUSE-Autoskill without skills achieves 40% (2 of 5 runs). The generated skill *adaptive-cruise-pid-controller* codifies the discrete PID equation, anti-windup, gain-tuning heuristics, and the JSON file format required by the verifier; Phase 2 accuracy reaches **100%**. Hermes using the same skill improves from 20% to 60%, confirming the skill transfers domain knowledge rather than memorizing the task.

(ii) *flink-query* asks the agent to author an Apache Flink Java job that reads gzipped Google Cluster-Data traces, performs microsecond event-time sessionization, and emits tuples in an exact format. The baseline solves only one of five runs (20%) because the agent cannot recover the project’s POJO and AppBase skeleton conventions from documentation alone within its turn budget. The generated skill *implement-clusterdata-flink-session-query* packages the schema parsing, the `clusterdata.utils.AppBase` extension protocol, event-time session triggers, and a Maven-based validation recipe with synthetic gzipped data; Phase 2 reaches **100%**.

(iii) *weighted-gdp-calc* requires filling an Excel workbook with two-condition lookups and SUMPRODUCT-based weighted means while preserving existing formatting and avoiding macros/VBA. The generated skill *excel-financial-formula-modeling* names `openpyxl` as the right tool, lists the formula patterns, and adds a verification step that recomputes target cells from source data; the baseline jumps from 20% to **100%**. Notably, the same skill description guides Hermes through the identical workflow without modification.

(iv) *Regression: hvac-control*. The largest regression (80% $\rightarrow$ 20%) occurs on a task that requires PI control of a first-order thermal simulator. The source trajectory used a calibration window and gain-estimation routine specific to that simulator’s noise profile; when re-applied in fresh runs, the variance in calibration data occasionally produces tuned gains outside the verifier’s stability margin. This is a case where the skill encodes a procedure that worked once but is less robust than baseline trial-and-error, and motivates the audit finding (next subsection) that some skills carry source-trajectory-specific assumptions that limit out-of-distribution robustness.

4.6 Analysis

Skill Quality Audit We manually inspect the 35 generated skills for potential benchmark leakage. None of the skills hardcode expected verifier outputs, branch on task identifiers, or read from ground-truth files. A

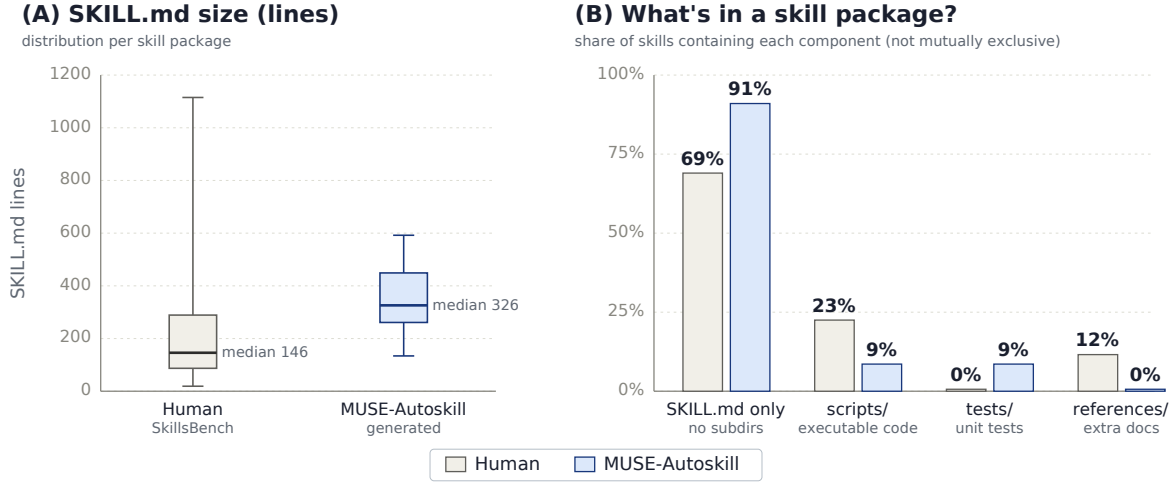


Figure 6 Skill anatomy: human-authored vs MUSE-generated. (A) SKILL.md line counts: MUSE skills are $\sim 2.2\times$ longer (median 326 vs. 146 lines) with a tighter IQR, reflecting more consistent, procedure-heavy structure. (B) Share of skill packages containing each subdirectory (not mutually exclusive). The dominant pattern in both groups is “SKILL.md only” (69% human, 91% MUSE). MUSE is the only group that ships `tests/`, by construction of the lifecycle.

subset of skills contain benchmark-specific assumptions such as fixed file names, directory paths, or numerical ranges derived from the source run. These do not constitute cheating but may limit generalization to out-of-distribution inputs. Cleaning such assumptions is a clear direction for future work, especially when deploying generated skills beyond the source benchmark.

Skill Anatomy: Human vs Generated Figure 6 compares the 35 MUSE-generated skills against 249 human-authored SkillsBench skills. MUSE skills carry a median SKILL.md of 326 lines (15.8 KB) vs. 146 lines (6.6 KB) for human skills, roughly $2.2\times$ longer. Inspection of the longer files indicates that the extra content is procedural rather than verbose: MUSE skills tend to spell out input/output schemas, failure modes, and step-by-step procedures that human authors leave implicit. The subdirectory composition also differs: 22% of human skills include `scripts/` but none include `tests/`, whereas MUSE includes `tests/` in 9% of packages (and the lifecycle gates registration on those tests passing). This means that, by construction, MUSE skills are testable as a system property rather than as an authoring convention.

Efficiency- and Cost-Quality Tradeoffs Figure 7 shows the same three agents under both conditions on two complementary axes. **Panel (A)** plots median per-task latency against mean reward: all three agents move toward higher reward without paying a latency cost, because the skill replaces ad-hoc reasoning with a pre-vetted procedure. We also measured ReAct turn counts: MUSE runs deeper loops (median 18–19 turns) than Codex (11–12) or Hermes (13–14), consistent with its higher reward at comparable wall-clock latency. **Panel (B)** re-plots the same points with median per-task tokens on the x-axis. Compared to the latency view, the picture is less of a free lunch: the arrows still go upward, but they also drift to the right, because the skill text is loaded into the prompt and the agent often invokes the skill sandbox tools on top of its base reasoning. The slope of each arrow (reward gained per extra token) is a more honest summary of the cost-benefit tradeoff than the latency arrow alone, and shows that even the most token-hungry agent, MUSE-Autoskill, recovers ~ 15 pp of reward for $\sim 12\%$ more tokens. Coupled with prompt caching (which absorbs about half of the marginal input cost; see Table 9), this puts the dollar cost of the skill-induced lift well below what the raw token deltas suggest.

Cost Distributions Figure 8 shows per-task latency (panel a) and total tokens (panel b) for all three agents under both conditions. Three patterns are worth flagging. (i) Hermes is the leanest on both axes (median latency 351–370 s, median tokens 163–172K), reflecting its shorter ReAct loop (13–14 turns) and tighter context.

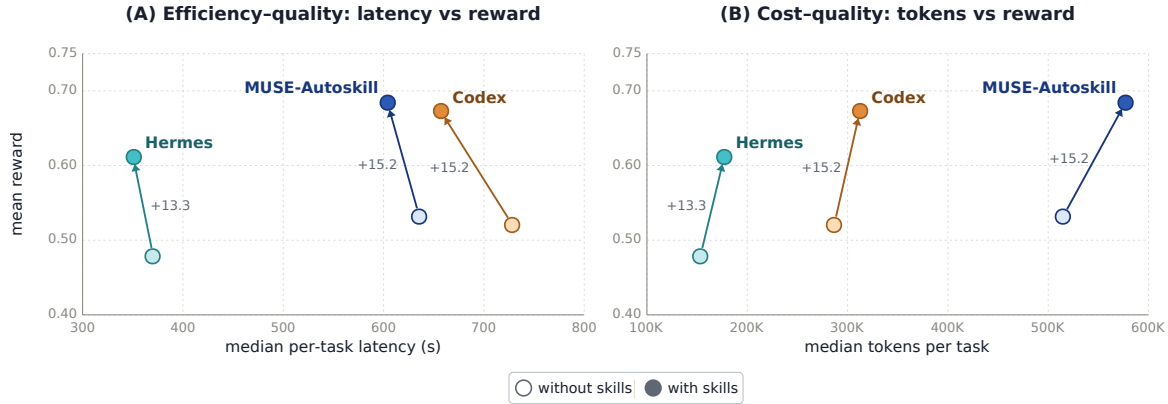


Figure 7 Skill-induced tradeoffs in two dimensions (3 agents $\times$ {without, with} skills). **(A) Latency vs reward.** Adding skills moves every agent up-and-left: higher reward at lower or unchanged latency, a Pareto improvement. **(B) Tokens vs reward.** The same shift becomes up-and-right since the skill text loads into the prompt. Reward gained per extra K token: ~ 0.56 pp/K (Hermes), ~ 0.58 pp/K (Codex), ~ 0.25 pp/K (MUSE). Prompt caching absorbs $\sim$ half of the marginal input cost (Table 9).

MUSE-Autoskill is the most token-heavy (515–577K, $\sim 1.8\times$ Codex’s 286–312K) due to its deeper 18–19-turn loop, but its IQR is narrower than Codex’s because adaptive context compression bounds per-turn prompt size. **(ii)** Adding human skills tightens the upper tail consistently—Codex’s p75 latency drops 18% (1297 $\rightarrow$ 1066 s) and p75 tokens drop 10% (595K $\rightarrow$ 537K)—supporting our claim that skills replace ad-hoc exploration with a pre-vetted procedure rather than shifting work elsewhere. **(iii)** Roughly half of MUSE-Autoskill’s and Codex’s input tokens are cached prompt prefixes (Table 9), so prompt-cache pricing absorbs a sizable share of the marginal cost of adding skills; output tokens stay $< 3\%$ of the total in every cell.

Bottleneck Analysis The 16 tasks where no skill is generated are predominantly from scientific computing and system operations, domains where the agent’s baseline performance is weakest. This suggests that the bottleneck is Phase 1 coverage rather than skill generation quality. Future improvements should focus on enhancing exploration in Phase 1 or on extracting diagnostic skills from partial or failed trajectories rather than waiting for a fully successful run.

5 Real-World Deployment and Impact

Beyond the controlled SkillsBench evaluation, the skill-centric design of MUSE-Autoskill is already being adopted in production systems, where skills serve as the common unit of capability shared across agents and users. SkillMarket exposes the skill-creation pipeline to end users, distilling a successful trajectory into a reusable, self-tested skill package without manual authoring; planned releases add skill management and updating, so that deployed skills can be versioned and refined as tasks and environments drift over time. ArkClaw integrates the skill-retrieval component as a find-skill capability, letting an agent locate the most relevant existing skill before synthesizing a new one, and a planned extension treats an entire agent as an invocable sub-agent, so that a single skill can encapsulate delegated multi-agent behavior.

SkillHub operationalizes the full skill lifecycle, covering creation, evaluation, memory, management, and refinement, as a hosted service that gives teams one place to store, evaluate, and govern skills together with their accumulated per-skill experience. Taken together, these deployments show that the lifecycle abstraction is not specific to our benchmark setting: the same retrieve-or-create decision, bundled tests, and per-skill memory carry over to systems built and used by different teams, and an improvement to a shared skill propagates to every agent and product that depends on it.

Looking forward, we expect skills to take on a broader role as the primitive for defining workflows. Rather than hand-wiring agent pipelines, developers will compose and version skills whose bundled tests and memory keep the resulting workflows self-documenting and easier to maintain. This shifts ongoing maintenance cost from

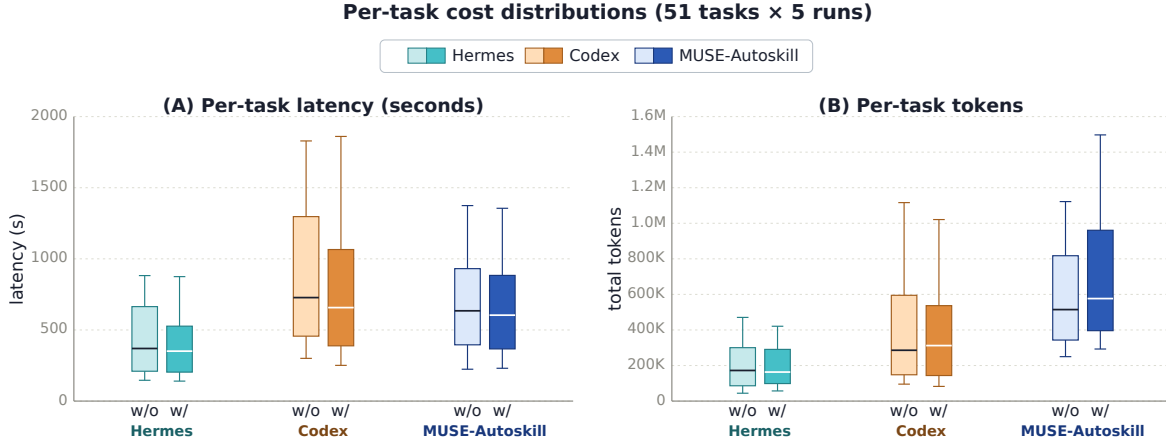


Figure 8 Per-task cost distributions (51 tasks × 5 runs). **(A)** agent execution latency in seconds; **(B)** total tokens (input + output, summed across all API calls). Within each agent, the lighter shade is without skills and the darker shade is with human skills. Boxes span the IQR (p25–p75), the centre line marks the median, and whiskers extend to p10 and p90. Hermes is roughly half as costly as MUSE-Autoskill on both axes. Adding human skills cuts median latency by 4–10% for every agent and barely changes median tokens (the skill text loads into the prompt but replaces longer reasoning), while consistently tightening the upper tail—e.g., Codex p75 latency drops −18% (1297→1066 s).

bespoke glue code to a shared, continuously evaluated skill ecosystem, and we view these early deployments as evidence that a unified skill lifecycle is a practical foundation for agents whose capabilities compound, rather than erode, as they are maintained at scale.

6 Conclusion

We present a skill-centric agent framework that enables agents to improve their task-solving capability by acquiring, reusing, and refining skills through a unified lifecycle. By representing all functionality as structured skills, the agent reduces redundant reasoning and improves efficiency over time. Our design integrates skill creation, evaluation, execution, memory, and management, supported by minimal built-in skills such as *skill_create* and *web_search*. Experiments on SkillsBench demonstrate that human skills reliably improve task accuracy, that MUSE-Autoskill can automatically generate high-quality skills from successful trajectories (reaching 87.94% on tasks with generated skills), and that generated skills transfer to other agents with minimal accuracy loss. The framework is already deployed in production systems that expose skill creation, discovery, and lifecycle management to real users (Section 5). Overall, this work provides a scalable approach toward agents with continuously evolving capabilities.

Limitations

Our evaluation covers 51 of 94 SkillsBench tasks; excluded tasks tend to have more complex Docker environments and may be harder, so reported numbers may overestimate system-wide performance. Skill generation succeeds on only 68.6% of tasks (35/51), and the generated skill for each task is distilled from a single source trajectory, which may not represent the most general solution path. Cross-agent transfer is validated only for the MUSE-Autoskill $\rightarrow$ Hermes direction; broader generalization across more agents remains to be confirmed. With 5 runs per task, confidence intervals for individual tasks are wide, particularly for binary-reward tasks.

A further concern is that, in our self-creation experiment, each skill is generated from one successful Phase 1 trajectory on a task and then re-evaluated on the same task for 5 runs. Although the verifier is deterministic and we do not feed task-specific ground-truth into the skill (Section 4), this protocol may still over-state the within-task gain by tightly coupling skill content to the source trajectory. In future work we plan to (i) run all 94 SkillsBench tasks rather than the 51-task subset; (ii) validate the framework with additional backbone models beyond GPT-5.5; and (iii) test on independent benchmarks beyond SkillsBench to assess generalisation of both the skill-generation pipeline and the cross-agent transfer claim under realistic deployment conditions.

References

- [1] Salaheddin Alzubi, Noah Provenzano, Jaydon Bingham, Weiyuan Chen, and Tu Vu. Evoskill: Automated skill discovery for multi-agent systems. [arXiv preprint arXiv:2603.02766](#), 2026.
- [2] Anthropic. Agent Skills: Equipping Agents for the Real World. <https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills>, 2025. Open standard released December 2025; <https://github.com/anthropics/skills>.
- [3] Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. Teaching large language models to self-debug. In The Twelfth International Conference on Learning Representations, ICLR, Vienna, Austria, 2024.
- [4] Hongcheol Cho, Ryangkyung Kang, and Youngeun Kim. Skillret: A large-scale benchmark for skill retrieval in llm agents. [arXiv preprint arXiv:2605.05726](#), 2026.
- [5] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. Metagpt: Meta programming for A multi-agent collaborative framework. In The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024, 2024.
- [6] Xu Huang, Weiwen Liu, Xiaolong Chen, Xingmei Wang, Hao Wang, Defu Lian, Yasheng Wang, Ruiming Tang, and Enhong Chen. Understanding the planning of LLM agents: A survey. [CoRR](#), abs/2402.02716, 2024.
- [7] Huiqiang Jiang, Qianhui Wu, Xufang Luo, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. Longllmlingua: Accelerating and enhancing llms in long context scenarios via prompt compression. In Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL, pages 1658–1677, Bangkok, Thailand, 2024. Association for Computational Linguistics.
- [8] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R. Narasimhan. Swe-bench: Can language models resolve real-world github issues? In The Twelfth International Conference on Learning Representations, ICLR, Vienna, Austria, 2024. OpenReview.net.
- [9] Xiangyi Li, Wenbo Chen, Yimin Liu, Shenghan Zheng, Xiaokun Chen, Yifeng He, Yubo Li, Bingran You, Haotian Shen, Jiankai Sun, et al. Skillsbench: Benchmarking how well agent skills work across diverse tasks. [arXiv preprint arXiv:2602.12670](#), 2026.
- [10] Xiaoxi Li, Wenxiang Jiao, Jiarui Jin, Shijian Wang, Guanting Dong, Jiajie Jin, Hao Wang, Yinuo Wang, Ji-Rong Wen, Yuan Lu, et al. Omnigaia: Towards native omni-modal ai agents. [CoRR](#), abs/2602.22897, 2026.
- [11] Huawei Lin, Yunzhi Shi, Tong Geng, Weijie Zhao, Wei Wang, and Ravender Pal Singh. Agent-omni: Test-time multimodal reasoning via model coordination for understanding anything. [CoRR](#), abs/2511.02834, 2025.
- [12] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. [Trans. Assoc. Comput. Linguistics](#), 12:157–173, 2024.

- [13] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. Agentbench: Evaluating llms as agents. In The Twelfth International Conference on Learning Representations, ICLR, Vienna, Austria, 2024. OpenReview.net.
- [14] Yuchen Ma, Yue Huang, Han Bao, Haomin Zhuang, Swadheen Shukla, Michel Galley, Xiangliang Zhang, and Stefan Feuerriegel. Skillgen: Verified inference-time agent skill synthesis. arXiv preprint arXiv:2605.10999, 2026.
- [15] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback. In Advances in Neural Information Processing Systems, NeurIPS, New Orleans, LA, 2023.
- [16] Grégoire Mialon, Clémentine Fourrier, Thomas Wolf, Yann LeCun, and Thomas Scialom. GAIA: a benchmark for general AI assistants. In The Twelfth International Conference on Learning Representations, ICLR, Vienna, Austria, 2024. OpenReview.net.
- [17] Siru Ouyang, Jun Yan, Yanfei Chen, Rujun Han, Zifeng Wang, Bhavana Dalvi Mishra, Rui Meng, Chun-Liang Li, Yizhu Jiao, Kaiwen Zha, et al. Skillos: Learning skill curation for self-evolving agents. arXiv preprint arXiv:2605.06614, 2026.
- [18] Charles Packer, Vivian Fang, Shishir G. Patil, Kevin Lin, Sarah Wooders, and Joseph E. Gonzalez. Memgpt: Towards llms as operating systems. CoRR, abs/2310.08560, 2023.
- [19] Joon Sung Park, Joseph C. O’Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology, UIST 3, pages 2:1–2:22, San Francisco, CA, 2023.
- [20] Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large language model connected with massive apis. In Advances in Neural Information Processing Systems, NeurIPS, Vancouver, BC, Canada, 2024.
- [21] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In Advances in Neural Information Processing Systems, NeurIPS, New Orleans, LA, 2023.
- [22] Samuel Schmidgall, Yusheng Su, Ze Wang, Ximeng Sun, Jialian Wu, Xiaodong Yu, Jiang Liu, Michael Moor, Zicheng Liu, and Emad Barsoum. Agent laboratory: Using LLM agents as research assistants. In Findings of the Association for Computational Linguistics: EMNLP 2025, Suzhou, China, November 4-9, 2025, pages 5977–6043, 2025.
- [23] Yongliang Shen, Kaitao Song, Xu Tan, Dongsheng Li, Weiming Lu, and Yueting Zhuang. Hugginggpt: Solving AI tasks with chatgpt and its friends in huggingface. CoRR, abs/2303.17580, 2023.
- [24] Yaorui Shi, Yuxin Chen, Zhengxi Lu, Yuchun Miao, Shugui Liu, Qi Gu, Xunliang Cai, Xiang Wang, and An Zhang. Skill1: Unified evolution of skill-augmented agents via reinforcement learning. arXiv preprint arXiv:2605.06130, 2026.
- [25] Yuchen Shi, Yuzheng Cai, Siqi Cai, Zihan Xu, Lichao Chen, Yulei Qin, Zhijian Zhou, Xiang Fei, Chaofan Qiu, Xiaoyu Tan, et al. Youtu-agent: Scaling agent productivity with automated generation and hybrid policy optimization. arXiv preprint arXiv:2512.24615, 2025.
- [26] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: language agents with verbal reinforcement learning. In Advances in Neural Information Processing Systems, NeurIPS, New Orleans, LA, 2023.
- [27] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. Trans. Mach. Learn. Res., 2024, 2024.
- [28] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, and et al. Openhands: An open platform for AI software

- developers as generalist agents. In The Thirteenth International Conference on Learning Representations, ICLR, Singapore, 2025. OpenReview.net.
- [29] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Shaokun Zhang, Erkang Zhu, Beibin Li, Li Jiang, Xiaoyun Zhang, and Chi Wang. Autogen: Enabling next-gen LLM applications via multi-agent conversation framework. CoRR, abs/2308.08155, 2023.
 - [30] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. In The Twelfth International Conference on Learning Representations, ICLR, Vienna, Austria, 2024. OpenReview.net.
 - [31] Renjun Xu and Yang Yan. Agent skills for large language models: Architecture, acquisition, security, and the path forward. arXiv preprint arXiv:2602.12430, 2026.
 - [32] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering. In Advances in Neural Information Processing Systems, NeurIPS, Vancouver, BC, Canada, 2024.
 - [33] Min Yang, Jinghua Piao, Xu Xia, Xiaochong Lan, Jiaju Chen, Yongshun Gong, and Yong Li. Skillmaster: Toward autonomous skill mastery in llm agents. arXiv preprint arXiv:2605.08693, 2026.
 - [34] Yutao Yang, Junsong Li, Qianjun Pan, Bihao Zhan, Yuxuan Cai, Lin Du, Jie Zhou, Kai Chen, Qin Chen, Xin Li, et al. Autoskill: Experience-driven lifelong learning via skill self-evolution. arXiv preprint arXiv:2603.01145, 2026.
 - [35] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R. Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In The Eleventh International Conference on Learning Representations, ICLR, Kigali, Rwanda, 2023.
 - [36] Kechi Zhang, Jia Li, Ge Li, Xianjie Shi, and Zhi Jin. Codeagent: Enhancing code generation with tool-integrated agent systems for real-world repo-level coding challenges. In Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2024, Bangkok, Thailand, August 11-16, 2024, pages 13643–13658, 2024.
 - [37] Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. Expel: LLM agents are experiential learners. In Thirty-Eighth AAAI Conference on Artificial Intelligence, AAAI, pages 19632–19642, Vancouver, Canada, 2024.
 - [38] Junhao Zheng, Xidi Cai, Qiuke Li, Duzhen Zhang, ZhongZhi Li, Yingying Zhang, Le Song, and Qianli Ma. Lifelongagentbench: Evaluating llm agents as lifelong learners. arXiv preprint arXiv:2505.11942, 2025.
 - [39] Shanshan Zhong, Yi Lu, Jingjie Ning, Yibing Wan, Lihan Feng, Yuyi Ao, Leonardo FR Ribeiro, Markus Dreyer, Sean Ammirati, and Chenyan Xiong. Skilllearnbench: Benchmarking continual learning methods for agent skill generation on real-world tasks. arXiv preprint arXiv:2604.20087, 2026.

A Selected Task List

Table 7 lists all 51 selected SkillsBench tasks used in our evaluation, together with their domain and whether MUSE-Autoskill successfully generated a skill in the automatic skill generation experiment (Section 4.3).

B Skill Package Schema

A skill is a self-contained directory rooted at the kebab-case skill name. The directory always contains a top-level `SKILL.md` written in Markdown with a YAML frontmatter block, and may optionally contain the subdirectories `scripts/`, `tests/`, `resources/`, and `references/`. Skills that do not need code consist of `SKILL.md` alone, which is the dominant pattern in practice (Figure 6 B). The skill identifier is taken from the directory name, and the same name must appear in the frontmatter `name` field; this redundancy lets a skill be moved or copied without breaking its identity. The schema deliberately mirrors Anthropic’s Agent Skills format [2] so that skills produced by MUSE-Autoskill can be loaded by any agent that already understands that format, without translation. The minimum viable skill file is:

```
---
name:      <kebab-case skill identifier; must match the directory name>
description: <one-paragraph natural-language description; this is what the
              agent reads when deciding whether to invoke the skill>
---

# <Skill title in Title Case>

## When to use
- Bullet list of triggering task types.

## Core principles
1. Numbered list of invariants the implementation must preserve.

## Recommended tools and libraries
- Concrete library names, CLI commands, or sandbox tools.

## Workflow
Step-by-step procedure the agent should follow at runtime.
```

Catalog routing. The frontmatter `description` field is the only piece of the skill that is surfaced eagerly: at the start of every task the runtime injects a YAML catalog of all available skills (each entry containing just `name` and `description`) into the agent’s system prompt. The body of `SKILL.md` is loaded only after the agent decides, via the `read_skill` tool, that the skill is worth pulling into context. This two-stage lookup keeps the per-call input cost flat in the size of the skill bank: a bank with 100 skills adds only ~5–10K tokens of catalog, not the ~500K tokens that loading every skill body would require.

Subdirectory conventions. The optional subdirectories follow strict per-name conventions, so the agent can rely on layout when reading the skill at runtime: `scripts/` holds executable code (Python, shell, Node) that the skill instructs the agent to run inside the sandbox; `tests/` is a pytest-compatible test suite that the evaluator runs to validate the skill before registering it into the bank (failed tests block registration; see Appendix E for the create-evaluate-register loop in detail); `resources/` holds passive auxiliary files (data tables, prompt fragments, reference documents) that the skill loads on demand at execution time; and `references/` (used by some human SkillsBench skills) holds reference documentation that the agent may read but is not expected to execute. A skill never embeds dependencies: it relies on the sandbox image (or runtime `pip install` via the terminal tool) for any packages it needs, which keeps the skill bank itself a pure-text artifact safe to version-control and ship as a tarball.

Skill-level memory. Alongside the on-disk skill, each skill gets a sibling `.memory.md` file (created lazily on first write) into which the agent appends notes, lessons, and usage observations across tasks. This file is the concrete realisation of the skill-level memory described in Section 3.2. It is intentionally outside the skill directory’s published surface area (the leading dot, and exclusion from any `.tar` the user might ship) so that transferring the skill does not transfer experience accumulated from prior runs; experience is per-agent.

C File-System Layout

This appendix documents the on-disk layout the agent assumes at runtime. Every path is configurable; we list the defaults so that an outside reader can understand where each component of a published trajectory came from.

Agent home directory. On the host the agent runtime defaults to `$HOME/.autoskill` (overridable via the `AUTOSKILL_HOME` environment variable). It is created on first launch and contains all persistent state that is not part of an individual session:

```
~/autoskill/
+-- skills/                                # the skill bank: one directory per skill
|   +-- pdf-form-update-redaction/
|       +-- SKILL.md                      # frontmatter + body
|       +-- .memory.md                   # per-skill memory (Section 3.4)
|       +-- scripts/                     # optional: executable code
|       +-- tests/                       # optional: pytest-compatible
|       +-- resources/                   # optional: data / docs
+-- csv-summarize/
|   +-- SKILL.md                         # the typical "doc-only" skill
+-- ...
+-- memory/
|   +-- long_term_memory/
|       +-- memory.md                    # cross-session notes, lessons learned
+-- sessions/                             # per-session workspaces (see below)
    +-- 2d9b1c67f73947c4863b26a45c5098a8/
    +-- ...
```

Per-session workspace. A new directory under `$AUTOSKILL_HOME/sessions/<session_id>/` is created for each task invocation. The session id is a UUID-like string that also names the directory; the runtime persists the agent’s complete state into this directory at the end of every session. Inside one session:

```
sessions/<session_id>/
+-- instruction.md                        # the task prompt the agent received
+-- submitted_inputs/                    # files supplied by the caller
+-- submitted_skillhub/                  # any human skills injected at task start
+-- result_output_files/                 # final artifacts the agent produced
+-- agent_message.md                     # final-answer text returned to the caller
+-- agent.stdout.txt                     # log stream incl. per-call token usage
+-- events.jsonl                         # one JSON event per tool call / turn
+-- memory.md                           # short-term (session-scoped) memory
+-- ctx_state.json                       # serialised AgentContext (for resume)
+-- profile.json                         # latency breakdown (setup, exec, verifier)
+-- run_meta.json                        # reward, turn count, model, ...
```

The most important files for reproducibility are `events.jsonl`, which contains a strictly-ordered stream of every plan / action / observation in the run (we use it for Appendix H), and `ctx_state.json`, the snapshot used by the cross-session resume mechanism (Section 3.4). `ctx_state.json` contains the full conversation DAG: every `ConversationNode` with its original input, the `compressed_input` (if Level-1 compression was

applied), and both pointer sets (`parent_id` for the active chain, `history_prev/history_next` for the original ordering).

Sandbox layout. Each invocation of `create_sandbox` spawns an isolated process with its own filesystem rooted at `/sandbox` (the exact backing depends on the sandbox factory: local processes, Docker containers, or a managed sandbox service all expose the same interface). Files the agent uploads via `sandbox_upload` land under `/sandbox/inputs/`; files produced by scripts go under `/sandbox/outputs/` and are pulled back with `sandbox_download` when the agent needs them. The sandbox is destroyed at the end of the session (or earlier if the agent explicitly calls `close_sandbox`), so no skill execution can affect host state.

Memory file format. All three memory files (long-term, short-term, and per-skill) share the same plain-Markdown format: an append-only writer appends a single block of the form

```
## 2026-05-07 10:34:33 UTC
<agent-written content, one short paragraph or list>
```

to the appropriate file. Read access is line-buffered and unparsed; the agent never edits or deletes existing entries, which keeps memory append-only and makes the file safe to read from multiple sessions concurrently.

D Hyperparameters and Runtime Configuration

Table 8 lists every runtime constant used in the SkillsBench experiments. All values were held fixed across all 51 tasks; we did not perform per-task tuning. The values fall into four groups, each with a specific design intent.

Compression thresholds. `COMPRESS_TOKEN_THRESHOLD` (180K) is set just below the model’s hard 200K context limit, leaving a ~10% headroom so a Level-2 compression call can itself fit in context. `NODE_COMPRESS_TOKEN_THRESHOLD` (15K) is the size at which a single tool output stops being amortisable across turns and starts dominating the prompt cache cost; below this threshold, leaving the original verbatim is preferable to summarising. `COMPRESS_KEEP_FIRST_TURNS` and `COMPRESS_KEEP_LAST_TURNS` (both 5) ensure that the task framing (the system prompt and the first few turns of grounding) and the immediate working context (the most recent five turns) are always sent verbatim; only the middle of the conversation is eligible for compression. In practice 5+5 turns are sufficient overhead even on the longest tasks we observed (max 69 turns).

Tool execution timeouts. The hierarchy `TOOL_TIMEOUT_SECONDS` (300) > `VERIFY_COMPLETION_TIMEOUT_SECONDS` (120) > `TERMINAL_TIMEOUT_SECONDS` (60) = `EXEC_CODE_TIMEOUT_SECONDS` (60) reflects the expected wall-clock cost of each operation: a generic tool (e.g. a multi-step skill invocation) may block for several minutes, the completion checker is bounded to a single LLM call plus diagnostics, and individual shell / Python snippets are kept short to keep the ReAct loop responsive. `MODEL_TIMEOUT_SECONDS` (300) is a guard against API hangs; on success the actual LLM call completes in 5–30 s. `TOOL_TEXT_LIMIT` (8,192 characters) is the hard cap on a single tool output before truncation, which protects the active chain from a single misbehaving tool dumping an entire log file.

Retry and verification. `MAX_RETRY` (5) is the per-call exponential-backoff budget for transient API failures (HTTP 429, 5xx). `VERIFY_COMPLETION_TURN_THRESHOLD` (4) is the smallest number of turns after which the agent is allowed to call `final_answer`; below this threshold a `verify_completion` pre-check is forced, which prevents the agent from prematurely terminating on tasks it has barely engaged with.

Backbone model, agent versions, and evaluation. All three agents share the same model id (GPT-5.5 (04/24/2026)) at provider defaults; we did not set temperature or top-*p*, so accuracy differences in Table 13 reflect agent-system design rather than sampling. The agent runtimes are **MUSE-Autoskill** (this work, running its own backend as described in Section 4); **Codex** CLI from the SkillsBench evaluation harness, using the same backbone through an OpenAI-style API endpoint; and **Hermes**, a third-party agent runtime, also configured against the same backbone via an OpenAI-compatible endpoint. The backbone model id is the only shared

identifier across the three runtimes; their internal prompts, tool definitions, and context handling differ. Every task is run 5 times in independent Docker containers; the SkillsBench harness controls per-task wall-clock budget.

Table 8 All runtime constants used in the experiments. The same values were used for MUSE-Autoskill, Codex, and Hermes (Codex and Hermes inherit only the model-level constants; their tool-execution and compression behaviour is governed by their own runtimes). Tasks were graded by the SkillsBench verifier in unmodified Docker environments.

Parameter	Value	Role
<i>Backbone model</i>		
model id	GPT-5.5 (04/24/2026)	shared across all three agents
temperature	default	no sampling override
<i>Context compression (see Appendix E)</i>		
COMPRESS_TOKEN_THRESHOLD	180,000	total-context trigger for Level-2
NODE_COMPRESS_TOKEN_THRESHOLD	15,000	per-node trigger for Level-1
COMPRESS_KEEP_FIRST_TURNS	5	oldest turns kept verbatim
COMPRESS_KEEP_LAST_TURNS	5	most recent turns kept verbatim
<i>Tool execution</i>		
TOOL_TEXT_LIMIT	8,192 chars	per-call tool output truncation
TOOL_TIMEOUT_SECONDS	300	generic tool deadline
TERMINAL_TIMEOUT_SECONDS	60	shell-command deadline
EXEC_CODE_TIMEOUT_SECONDS	60	Python-snippet deadline
VERIFY_COMPLETION_TIMEOUT_SECONDS	120	deadline for the completion checker
MODEL_TIMEOUT_SECONDS	300	deadline for a single LLM call
MAX_RETRY	5	per-call retry budget on API failures
VERIFY_COMPLETION_TURN_THRESHOLD	4	turns after which <code>final_answer</code> requires verification
<i>Evaluation protocol</i>		
runs per task	5	independent Docker containers
timeout per task	inherited from SkillsBench harness	varies by task

E Compression Algorithm

Context compression is invoked by `maybe_compress_history(ctx, model)` at the start of every ReAct turn, immediately after the agent’s response is appended to the conversation and before the next LLM call is issued. The function returns silently when the active chain is under budget (which is the common case at the start of a run) and only triggers an LLM-summarisation call when the total token estimate crosses `COMPRESS_TOKEN_THRESHOLD`. We implement two levels of progressively more aggressive compression; in our SkillsBench runs Level-1 is sufficient for the vast majority of contexts that exceed the budget and Level-2 fires only on the longest-running tasks (turn count >50). Both levels operate exclusively on the active chain (the linked list reachable via `parent_id`); the immutable `history_prev/history_next` pointers are never rewritten, so any prior state can still be reconstructed for cross-session resume or for post-hoc trajectory analysis. The high-level control flow is:

```
def maybe_compress_history(ctx, model):
    chain = walk(parent_id from tip to root)
    total_tokens = sum(estimate_tokens(node) for node in chain)
    if total_tokens <= COMPRESS_TOKEN_THRESHOLD:
        return # under budget; nothing to do

    # ---- Level 1: per-node, in-place summary on oversized nodes ----
    # never touch the first K=5 or last K=5 turns
    middle = chain[KEEP_FIRST : -KEEP_LAST]
    for node in middle:
        if estimate_tokens(node) > NODE_COMPRESS_TOKEN_THRESHOLD:
            summary = model.summarize(node.input + node.model_output)
            node.compressed_input = summary
            node.is_node_compressed = True # reads return summary

    if recompute_total(chain) <= COMPRESS_TOKEN_THRESHOLD:
```

```

        return                                                    # Level 1 was enough

# ---- Level 2: collapse the middle span into one summary node ----
span    = chain[KEEP_FIRST : -KEEP_LAST]
summary = model.summarize(concat(span))
sNode   = new ConversationNode(
    is_summary      = True,
    parent_id       = chain[KEEP_FIRST - 1].node_id,
    compressed_input = summary,
)
chain[-KEEP_LAST].parent_id = sNode.node_id # rewire chain

```

Cost. Compression itself costs LLM calls: Level-1 issues at most one summarisation call per oversized node, Level-2 issues exactly one summarisation call per trigger. Because the threshold is much larger than a typical tool output, the amortised cost is small (one extra LLM call every ~ 10 – 20 ReAct turns on the long-running tasks we observed). The summary calls use the same backbone model (GPT-5.5 (04/24/2026)) at provider defaults; we did not separately tune them.

Audit trail. The original `node.input` field is never mutated. Reads through the active-chain reader return `compressed_input` when `is_node_compressed` is True, so the active chain shrinks; reads through the full-history reader ignore the `compressed_input` field and walk the immutable history pointers, so the original ordering is recoverable. Level-2’s synthetic summary node has `is_summary=True` and, by construction, no history pointers, so the full-history reader skips it and recovers exactly the original sequence of turns. This means any compressed run can be “replayed” for analysis without re-running the agent.

Why not just truncate? A simpler alternative (drop the oldest middle turns when the budget is exceeded) would lose information silently. Our preliminary experiments showed that on multi-step tasks the agent revisits early-context facts roughly 30–40% of the time (e.g. to recheck an input filename or recall a parsing-format detail); truncation forced wasteful re-discovery. Summarisation preserves these facts at $\sim 1/10$ the token cost, which is the regime where the LLM-call overhead pays for itself.

F Detailed Token Breakdown

Figure 8(B) reports per-task token distributions as box plots. Table 9 below splits each agent’s input tokens into the fresh component (paid at full rate by the model provider) and the cached component (paid at a discounted prompt-cache rate), and reports output and reasoning tokens separately. Numbers are medians over $51 \text{ tasks} \times 5 \text{ runs}$.

A few patterns are worth flagging. (i) Roughly half of every agent’s input is cached: $294,400/503,066 = 58.5\%$ for MUSE-Autoskill without skills, $326,784/557,880 = 58.6\%$ with skills; $157,824/267,783 = 58.9\%$ and $149,504/276,371 = 54.1\%$ for Codex; $227,584/389,011 = 58.5\%$ and $178,688/334,972 = 53.3\%$ for Hermes. This is because all three agents reuse the same system prompt and tool definitions on every turn, so the provider’s prompt cache absorbs the constant prefix. (ii) Output is consistently $< 3\%$ of the total: 12K vs 503K input for MUSE-Autoskill, 7K vs 268K for Codex, 8K vs 389K for Hermes. (iii) Adding skills costs about 20–25K extra fresh input tokens per task for MUSE-Autoskill and Codex (the skill catalog plus any `SKILL.md` content the agent loads), and reduces neither output nor reasoning. The reasoning component (which the model provider counts within “output”) is itself small (~ 4 K for MUSE-Autoskill, ~ 2 – 3 K for Hermes, ~ 2 K for Codex), consistent with the model emitting only short reasoning blocks per turn rather than long chain-of-thought traces.

Table 9 Per-task token usage (median across 51 tasks $\times$ 5 runs). “Fresh in” is the portion of input tokens that hit the model’s cold path; “cached in” is the portion served from the provider’s prompt cache at reduced cost. “Reasoning” is counted within “output” by the model API.

Agent	Condition	Fresh in	Cached in	Output	Reasoning	Total (in+out)
Codex	without skills	109,959	157,824	6,988	2,145	285,836
Codex	with skills	126,867	149,504	6,176	1,617	312,173
Hermes	without skills	161,427	227,584	8,344	3,026	172,180
Hermes	with skills	156,284	178,688	7,108	2,048	163,272
MUSE-Autoskill	without skills	208,666	294,400	11,998	4,347	514,642
MUSE-Autoskill	with skills	231,096	326,784	11,712	4,148	576,503

G Per-Domain Accuracy with Standard Deviation

Table 10 reports the per-domain mean accuracy together with task-level standard deviation, for each of the three agents and both skill conditions. Standard deviation is computed across the per-task means (each task’s mean is itself averaged over 5 runs); a high σ signals that the agent’s accuracy varies considerably across tasks in that domain (some tasks land near 100%, others near 0%) and does not directly reflect run-to-run noise.

Three patterns emerge consistently across all three agents. **First**, Sci & Eng shows the largest and most reliable benefit from skills (lifts of +17 to +24 percentage points), and is the only domain where the with-skills σ shrinks for all three agents (e.g. Codex drops from $\sigma=46.1$ to $\sigma=35.8$, Hermes from 47.0 to 32.6). This is consistent with a model where the skill provides the missing recipe for an otherwise-uniform difficulty distribution. **Second**, Document Processing already enjoys the highest without-skills baselines (71–82%), so the headroom is small: lifts compress to +2 to +11 pp and σ stays low throughout. By contrast, Ops & Planning has the lowest baselines (36–40%) but skills still recover +14 to +17 pp; inspection of the failed tasks in this domain (Appendix I) shows the remaining bottleneck is not skill quality but task feasibility, since many of these tasks involve obscure production tooling where neither baseline knowledge nor a one-page skill can substitute for hands-on debugging. **Third**, MUSE-Autoskill leads the with-skills column in 3 of 4 domains (Codex leads Sci & Eng by 5.7 pp), confirming that the gains in Figure 1 are not concentrated in one easy domain.

Table 10 Per-domain accuracy (%) with task-level standard deviation. Lift is the difference of means (w/ human skills – w/o skills). **Bold** marks MUSE-Autoskill entries that lead the with-skills column in each domain.

Agent	Domain	n tasks	w/o skills	w/ human skills	Lift
Codex	Sci & Eng	14	55.7 $\pm$ 46.1	78.6 $\pm$ 35.8	+22.9
Codex	Data Analysis	15	44.6 $\pm$ 42.4	60.2 $\pm$ 42.1	+15.6
Codex	Document Processing	9	82.2 $\pm$ 17.5	84.4 $\pm$ 20.6	+2.2
Codex	Ops & Planning	13	36.0 $\pm$ 35.0	51.4 $\pm$ 39.4	+15.4
Hermes	Sci & Eng	14	48.6 $\pm$ 47.0	72.9 $\pm$ 32.6	+24.3
Hermes	Data Analysis	15	43.6 $\pm$ 44.0	47.4 $\pm$ 40.4	+3.8
Hermes	Document Processing	9	71.1 $\pm$ 32.8	82.2 $\pm$ 29.0	+11.1
Hermes	Ops & Planning	13	36.0 $\pm$ 39.2	50.1 $\pm$ 40.8	+14.1
MUSE-Autoskill	Sci & Eng	14	55.7 $\pm$ 42.2	72.9 $\pm$ 41.1	+17.1
MUSE-Autoskill	Data Analysis	15	47.4 $\pm$ 44.6	61.8 $\pm$ 44.9	+14.4
MUSE-Autoskill	Document Processing	9	77.8 $\pm$ 25.7	88.9 $\pm$ 19.1	+11.1
MUSE-Autoskill	Ops & Planning	13	40.2 $\pm$ 40.8	57.1 $\pm$ 38.4	+16.9

H Latency and Turn-Count Distribution

Table 11 reports the percentile distribution of per-task latency (agent execution time in seconds, excluding the SkillsBench verifier) and ReAct turn counts. Both are computed across 51 tasks $\times$ 5 runs = 255 runs

per (agent, condition) cell. The three agents occupy three distinct operating points. **Hermes** is the fastest with the lightest reasoning budget: 90% of its tasks finish in under 900s, and its median turn count (13–14) is lower than MUSE-Autoskill’s. **Codex** sits in the middle with the longest tail; its 90th percentile latency exceeds 1,800s on hard tasks, even though its median (657–728s) is comparable to MUSE-Autoskill’s. This tail is driven by Codex’s tendency to re-run long shell commands on repeated retries when a tool call fails. **MUSE-Autoskill** runs the deepest loops (median 18–19 turns, max 69), but its latency tail is shorter than Codex’s ($p_{90} \leq 1,400s$) because compressed contexts and bounded tool outputs keep each turn fast.

A second observation is that adding skills strictly reduces median latency for every agent, by 4–10%. This is the temporal counterpart of the reward-gain shown in Figure 7(A): when a skill replaces ad-hoc reasoning, both wall-clock time and ReAct depth go down on average, while the success rate goes up.

Table 11 Distribution of per-task agent latency (seconds) and ReAct turn counts, by agent and skill condition. “p10” / “p25” / “p75” / “p90” are the 10th, 25th, 75th, and 90th percentile. “max” for turns is the absolute maximum observed across all runs in that cell.

Agent	Cond.	Latency (s)					Turns			
		p10	p25	median	p75	p90	p25	median	p75	max
Codex	w/o	293.9	455.2	727.6	1297.7	1837.6	8	11	17	48
Codex	w/	249.0	385.8	657.2	1067.4	1877.5	8	12	18	47
Hermes	w/o	144.0	209.4	369.7	665.8	897.7	10	14	19	73
Hermes	w/	140.4	202.3	351.1	527.4	883.3	10	13	18	61
MUSE-Autoskill	w/o	222.8	394.4	634.6	935.6	1378.2	14	18	24	69
MUSE-Autoskill	w/	229.3	365.8	604.1	885.1	1366.3	14	19	23	41

I Skill-Generation Failures: The 16 Uncovered Tasks

Table 12 lists the 16 of 51 tasks where Phase 1 produced no successful trajectory and consequently no skill was generated for Phase 2 self-creation. These tasks contribute 0% to the 51-task average in Table 4 but pass through Phase 2 of MUSE-Autoskill self-creation without modification (no skill is loaded, no skill is created); they remain available to all other phases. We list them here because the location of the failures, more than their count, characterises the limits of inference-time skill synthesis.

The failures concentrate in two patterns. The first is specialised production tooling, which dominates Ops & Planning (6 of 10 tasks in the domain fail): tasks like **azure-bgp-oscillation-route-leak** or **dapt-intrusion-detection** require the agent to understand vendor-specific telemetry, custom log formats, and remediation workflows that simply are not present in a general pretrained LLM’s knowledge. The same root cause shows up in two Document Processing failures (**llm-prefix-cache-replay**, which assumes intimate knowledge of an LLM serving stack, and **sec-financial-report**, which requires regulatory-specific filing schemas). The second pattern is numerically-heavy non-textual reasoning: tasks in Sci & Eng (**earthquake-plate-calculation**, **energy-unit-commitment**, etc.) and Data Analysis (**xlsx-recover-data**, **threejs-structure-parser**, etc.) where solving the task requires either a long numerical pipeline or robust parsing of an unfamiliar binary / structured format. In both patterns the bottleneck is Phase 1’s success rate, not the quality of the resulting skill: no successful trajectory exists from which to distill a skill in the first place. A natural next direction is to extract partial skills from failed trajectories (capturing the diagnostic moves that did work, even when the run ultimately ended at reward 0), which we leave to future work.

J Per-Task Accuracy: All Agents and Configurations

Table 13 reports accuracy (%) for all 51 tasks across every agent and configuration. **Bold** indicates the best result in each row. Subscript deltas (_{+x} / _{-x}) show the change relative to each agent’s own without-skills

baseline. MUSE-Autoskill columns are highlighted in blue.

Table 7 All 51 selected SkillsBench tasks. “n/a” indicates Phase 1 produced no successful run and no skill was generated; those tasks contribute 0% to the 51-task average in Table 4.

#	Task ID	Domain	Skill Generated
1	3d-scan-calc	Science & Engineering	✓
2	ada-bathroom-plan-repair	Document Processing	✓
3	adaptive-cruise-control	Science & Engineering	✓
4	azure-bgp-oscillation-route-leak	Ops & Planning	n/a
5	bike-rebalance	Ops & Planning	✓
6	citation-check	Document Processing	✓
7	civ6-adjacency-optimizer	Ops & Planning	✓
8	court-form-filling	Document Processing	✓
9	dapt-intrusion-detection	Ops & Planning	n/a
10	data-to-d3	Data Analysis	n/a
11	dialogue-parser	Data Analysis	✓
12	dynamic-object-aware-egomotion	Science & Engineering	n/a
13	earthquake-plate-calculation	Science & Engineering	n/a
14	econ-detrending-correlation	Data Analysis	✓
15	edit-pdf	Document Processing	✓
16	energy-market-pricing	Ops & Planning	✓
17	energy-unit-commitment	Ops & Planning	n/a
18	enterprise-information-search	Ops & Planning	n/a
19	financial-modeling-qa	Data Analysis	✓
20	fix-erlang-ssh-cve	Ops & Planning	✓
21	flink-query	Ops & Planning	✓
22	flood-risk-analysis	Science & Engineering	n/a
23	gravitational-wave-detection	Science & Engineering	✓
24	grid-dispatch-operator	Ops & Planning	n/a
25	hvac-control	Science & Engineering	✓
26	invoice-fraud-detection	Data Analysis	✓
27	jax-computing-basics	Science & Engineering	✓
28	lab-unit-harmonization	Data Analysis	✓
29	lake-warming-attribution	Science & Engineering	n/a
30	lean4-proof	Science & Engineering	✓
31	llm-prefix-cache-replay	Ops & Planning	n/a
32	mars-clouds-clustering	Science & Engineering	✓
33	multilingual-video-dubbing	Document Processing	✓
34	offer-letter-generator	Document Processing	✓
35	organize-messy-files	Ops & Planning	✓
36	paper-anonymizer	Document Processing	✓
37	parallel-tfidf-search	Data Analysis	✓
38	pdf-excel-diff	Document Processing	✓
39	powerlifting-coef-calc	Data Analysis	✓
40	pptx-reference-formatting	Document Processing	✓
41	protein-expression-analysis	Science & Engineering	✓
42	r2r-mpc-control	Science & Engineering	✓
43	radar-vital-signs	Science & Engineering	✓
44	reserves-at-risk-calc	Data Analysis	n/a
45	sales-pivot-analysis	Data Analysis	n/a
46	sec-financial-report	Data Analysis	n/a
47	spring-boot-jakarta-migration	Ops & Planning	✓
48	threejs-structure-parser	Data Analysis	n/a
49	threejs-to-obj	Data Analysis	✓
50	weighted-gdp-calc	Data Analysis	✓
51	xlsx-recover-data	Data Analysis	n/a

Table 12 The 16 SkillsBench tasks for which MUSE-Autoskill produced no successful Phase 1 trajectory, so no skill could be distilled. Causes are inferred from the agent’s stdout traces (no reverse-engineering of the verifier); they cluster around tasks that require either specialised production tooling or numerically-heavy non-textual reasoning.

Domain	Task ID	Likely failure cause
Ops & Planning	azure-bgp-oscillation-route-leak	specialised network-routing diagnosis
Ops & Planning	dapt-intrusion-detection	temporal anomaly detection in logs
Ops & Planning	data-to-d3	interactive D3.js visualisation
Ops & Planning	dynamic-object-aware-egomotion	3D vision / motion estimation
Ops & Planning	enterprise-information-search	query design over unstructured corpora
Ops & Planning	flood-risk-analysis	geospatial hydrology modelling
Sci & Eng	earthquake-plate-calculation	numerical PDE / plate-tectonics
Sci & Eng	energy-unit-commitment	MILP scheduling
Sci & Eng	grid-dispatch-operator	power-grid optimisation
Sci & Eng	lake-warming-attribution	climate attribution statistics
Data Analysis	reserves-at-risk-calc	financial risk modelling
Data Analysis	sales-pivot-analysis	spreadsheet pivot semantics
Data Analysis	threejs-structure-parser	parsing 3D scene formats
Data Analysis	xlsx-recover-data	recovering corrupted Excel files
Doc. Processing	llm-prefix-cache-replay	LLM internals trace replay
Doc. Processing	sec-financial-report	long-form regulatory parsing

Table 13 Per-task accuracy (%) for all 51 tasks. **Bold** = best in row. Subscript delta = change vs. same agent’s without-skills baseline (^{green} = improvement, ^{red} = regression). **Blue** columns = MUSE-Autoskill (ours). “Gen. Skills” = Hermes with MUSE-generated skills; “Self-Created” = MUSE Phase 2 (0.0% by definition where Phase 1 failed).

#	Task ID	Codex			Hermes			MUSE-Autoskill (Ours)		
		w/o Skills	Human Skills	w/o Skills	Human Skills	Gen. Skills	w/o Skills	Human Skills	Self-Created	
1	3d-scan-calc	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	
2	ada-bathroom-plan-repair	80.0	100.0 ^{+20.0}	80.0	80.0	100.0 ^{+20.0}	80.0	100.0 ^{+20.0}	100.0 ^{+20.0}	
3	adaptive-cruise-control	20.0	80.0 ^{+60.0}	20.0	80.0 ^{+60.0}	60.0 ^{+40.0}	40.0	80.0 ^{+40.0}	100.0 ^{+60.0}	
4	azure-bgp-oscillation-route-leak	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	
5	bike-rebalance	40.0	60.0 ^{+20.0}	80.0	100.0 ^{+20.0}	60.0 ^{+20.0}	60.0	60.0	40.0 ^{+20.0}	
6	citation-check	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	
7	civ6-adjacency-optimizer	28.0	88.0 ^{+60.0}	28.0	71.0 ^{+43.0}	70.0 ^{+42.0}	42.0	82.0 ^{+40.0}	70.0 ^{+28.0}	
8	court-form-filling	60.0	40.0 ^{+20.0}	20.0	20.0	100.0 ^{+80.0}	80.0	80.0	100.0 ^{+20.0}	
9	dapt-intrusion-detection	0.0	80.0 ^{+80.0}	0.0	60.0 ^{+60.0}	0.0	0.0	80.0 ^{+80.0}	0.0	
10	data-to-d3	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	
11	dialogue-parser	80.0	83.3 ^{+3.3}	83.3	83.3	83.3	83.3	86.6 ^{+3.3}	83.3	
12	dynamic-object-aware-egomotion	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	
13	earthquake-plate-calculation	0.0	100.0 ^{+100.0}	0.0	80.0 ^{+80.0}	0.0	0.0	100.0 ^{+100.0}	0.0	
14	econ-detrending-correlation	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	
15	edit-pdf	80.0	80.0	100.0	100.0	100.0	60.0	100.0 ^{+40.0}	100.0 ^{+40.0}	
16	energy-market-pricing	40.0	40.0	80.0	40.0 ^{+40.0}	100.0 ^{+20.0}	80.0	40.0 ^{+40.0}	100.0 ^{+20.0}	
17	energy-unit-commitment	0.0	0.0	0.0	20.0 ^{+20.0}	0.0	0.0	0.0	0.0	
18	enterprise-information-search	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	
19	financial-modeling-qa	40.0	80.0 ^{+40.0}	40.0	60.0 ^{+20.0}	100.0 ^{+60.0}	100.0	100.0	100.0	
20	fix-erlang-ssh-cve	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	
21	flink-query	40.0	0.0 ^{+40.0}	20.0	0.0 ^{+20.0}	100.0 ^{+80.0}	20.0	0.0 ^{+20.0}	100.0 ^{+80.0}	
22	flood-risk-analysis	0.0	40.0 ^{+40.0}	0.0	0.0	0.0	0.0	0.0	0.0	
23	gravitational-wave-detection	100.0	100.0	60.0	80.0 ^{+20.0}	100.0 ^{+40.0}	100.0	100.0	100.0	
24	grid-dispatch-operator	20.0	40.0 ^{+20.0}	0.0	0.0	0.0	20.0	40.0 ^{+20.0}	0.0 ^{+20.0}	
25	hvac-control	80.0	80.0	0.0	100.0 ^{+100.0}	40.0 ^{+40.0}	40.0	0.0 ^{+40.0}	40.0	
26	invoice-fraud-detection	0.0	0.0	0.0	0.0	0.0	100.0	100.0	100.0	
27	jax-computing-basics	0.0	0.0	100.0	60.0 ^{+60.0}	65.0 ^{+14.1}	67.5	100.0 ^{+32.5}	64.6 ^{+2.9}	
28	lab-unit-harmonization	69.6	100.0 ^{+30.4}	50.9	87.5 ^{+36.6}	65.0 ^{+14.1}	0.0	0.0	0.0	
29	lake-warming-attribution	0.0	100.0 ^{+100.0}	0.0	80.0 ^{+80.0}	0.0	0.0	0.0	0.0	
30	lean4-proof	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	
31	llm-prefix-cache-replay	20.0	100.0 ^{+80.0}	0.0	100.0 ^{+100.0}	0.0	0.0	100.0	0.0	
32	mars-clouds-clustering	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	
33	multilingual-video-dubbing	60.0	100.0 ^{+40.0}	20.0	40.0 ^{+20.0}	60.0 ^{+40.0}	60.0	40.0 ^{+20.0}	80.0 ^{+20.0}	
34	offer-letter-generator	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	
35	organize-messy-files	80.0	60.0 ^{+20.0}	80.0	60.0 ^{+20.0}	100.0 ^{+20.0}	100.0	100.0	100.0	
36	paper-anonymizer	60.0	60.0	40.0	100.0 ^{+60.0}	0.0 ^{+40.0}	20.0	80.0 ^{+60.0}	40.0 ^{+20.0}	
37	parallel-tfidf-search	100.0	100.0	80.0	80.0	100.0 ^{+20.0}	100.0	100.0	100.0	
38	pdf-excel-diff	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	
39	powerlifting-coef-calc	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	
40	pptx-reference-formatting	100.0	80.0 ^{+20.0}	80.0	100.0 ^{+20.0}	100.0 ^{+20.0}	100.0	100.0	100.0	
41	protein-expression-analysis	80.0	100.0 ^{+20.0}	100.0	100.0	100.0	20.0	100.0 ^{+80.0}	60.0	
42	r2r-mpc-control	100.0	100.0	100.0	60.0 ^{+40.0}	80.0 ^{+20.0}	60.0	100.0 ^{+40.0}	60.0	
43	radar-vital-signs	100.0	100.0	0.0	80.0 ^{+80.0}	20.0 ^{+20.0}	80.0	40.0 ^{+40.0}	80.0	
44	reserves-at-risk-calc	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	
45	sales-pivot-analysis	20.0	20.0	0.0	0.0	0.0	0.0	0.0	0.0	
46	sec-financial-report	0.0	60.0 ^{+60.0}	0.0	20.0 ^{+20.0}	0.0	0.0	100.0 ^{+100.0}	0.0	
47	spring-boot-jakarta-migration	100.0	100.0	80.0	100.0 ^{+20.0}	100.0 ^{+20.0}	100.0	100.0	100.0	
48	threejs-structure-parser	0.0	60.0 ^{+60.0}	0.0	40.0 ^{+40.0}	0.0	0.0	60.0 ^{+60.0}	0.0	
49	threejs-to-obj	100.0	100.0	100.0	100.0	100.0	100.0	100.0	100.0	
50	weighted-gdp-calc	60.0	100.0 ^{+40.0}	100.0	40.0 ^{+60.0}	100.0	20.0	80.0 ^{+60.0}	100.0 ^{+80.0}	
51	xlsx-recover-data	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	
Macro-average		52.11	67.28 ^{+15.2}	47.89	61.21 ^{+13.3}	58.40 ^{+10.5}	53.19	68.40 ^{+15.2}	60.35 ^{+7.2}	